

Homework 1 Report

Hande Eryilmaz, 21902678

Use of Generative AI Tools

Generative AI (ChatGPT, OpenAI GPT-5 architecture, 2026 version) was used to assist with clarifying homework requirements, reviewing theoretical concepts (Fourier transforms, MRI reconstruction, partial k-space methods, and phase compensation), and improving the clarity of the written report. It was also used to help interpret intermediate results and debug MATLAB implementations.

All MATLAB code was implemented, executed, and validated independently. All figures, numerical results, and execution logs were generated through personal MATLAB runs. I remain fully responsible for the correctness, originality, and interpretation of all submitted material and can explain and justify all code, results, and written content.

Preparation

Preparation 1: Centered FFT Implementation

Objective

In MRI reconstruction and image processing, the Fourier origin (DC component) is conventionally displayed at the center of the image. However, MATLAB's built-in `fft2` and `ifft2` functions assume that the zero-frequency component is located at the upper-left corner of the array. Therefore, a centered FFT implementation is required.

Theory

For a 2D discrete image $m(x,y)$, the centered Fourier transform is computed as:

$$D(k_x, k_y) = \text{fftshift} \{ \text{fft2} (\text{ifftshift} \{ m(x, y) \}) \}$$

Similarly, the centered inverse transform is:

$$m(x, y) = \text{fftshift} \{ \text{ifft2} (\text{ifftshift} \{ D(k_x, k_y) \}) \}$$

- `ifftshift` rearranges the quadrants to move the center to the origin before FFT.
- `fftshift` moves the zero-frequency component to the center for display.

This convention is used throughout the homework.

MATLAB Implementation

- `fft2c.m`
- `ifft2c.m`

```
function d = fft2c(im)
% d = fft2c(im)
% Performs centered 2D FFT
d = fftshift(fft2(ifftshift(i
m)));
end
```

```
function im = ifft2c(d)
% im = ifft2c(d)
% Performs centered 2D inverse
FFT
im = fftshift(ifft2(ifftshift
(d)));
end
```

- `fftc.m`

```
function d = fftc(im)
% d = fftc(im)
% Performs centered 1D FFT
d = fftshift(fft(ifftshift(i
m)));
end
```

- `ifftc.m`

```
function im = ifftc(d)
% im = ifftc(d)
% Performs centered 1D inverse
FFT
im = fftshift(ifft(ifftshift
(d)));
end
```

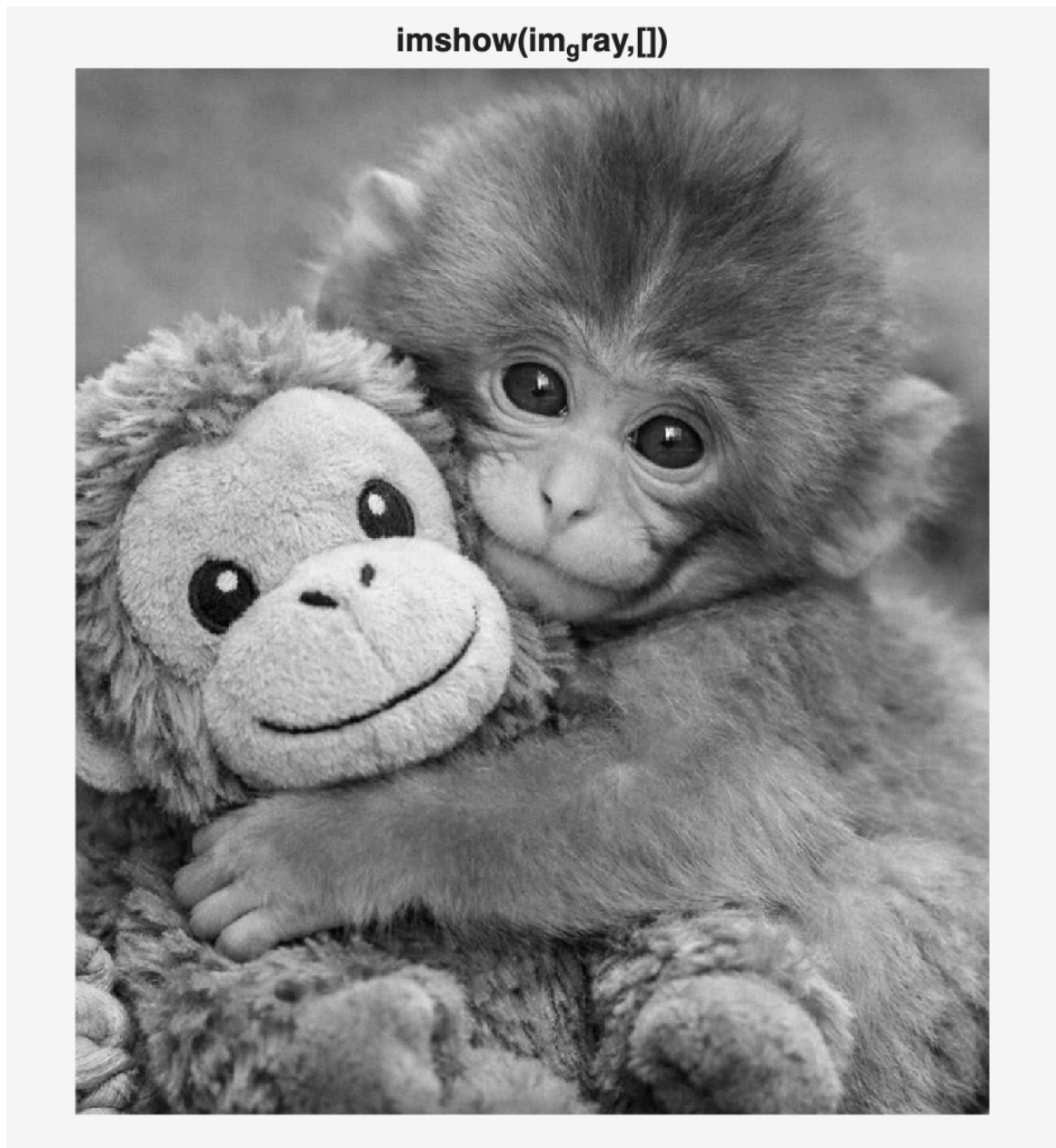


Figure 1: Preperation 1

Preparation 2: Displaying Images in MATLAB

Objective

To correctly visualize grayscale images and understand how intensity scaling affects image appearance in MATLAB.

Theory

The MATLAB function `imshow` displays images according to a specified intensity range.

By default:

```
imshow(im)
```

maps:

- Pixel value 0 → black
- Pixel value 1 → white

However, medical images and reconstructed images often have arbitrary dynamic ranges. Therefore, intensity normalization is required.

The recommended usage is:

```
imshow(im, [])
```

The empty brackets [] instruct MATLAB to:

- Map the **minimum value** of the image to black
- Map the **maximum value** of the image to white

This is equivalent to:

$$\text{imshow} \left(\frac{im - \min(im)}{\max(im) - \min(im)} \right)$$

or

```
imshow(im, [min(im(:)) max(im(:))])
```

Different intensity windows produce different visual contrast, which is important in MRI visualization.

MATLAB Implementation

- Example commands used during preparation:

```
imshow(im, [])
```

- Equivalent forms:

```
imshow((im - min(im(:))) / (max(im(:)) - min(im(:))))  
imshow(im, [min(im(:)) max(im(:))])
```

- Testing alternative intensity ranges:

```
imshow(im, [min(im(:))/2 max(im(:))/2])  
imshow(im, [0 max(im(:))/10])
```

These experiments demonstrated how windowing strongly affects image contrast and perceived detail.

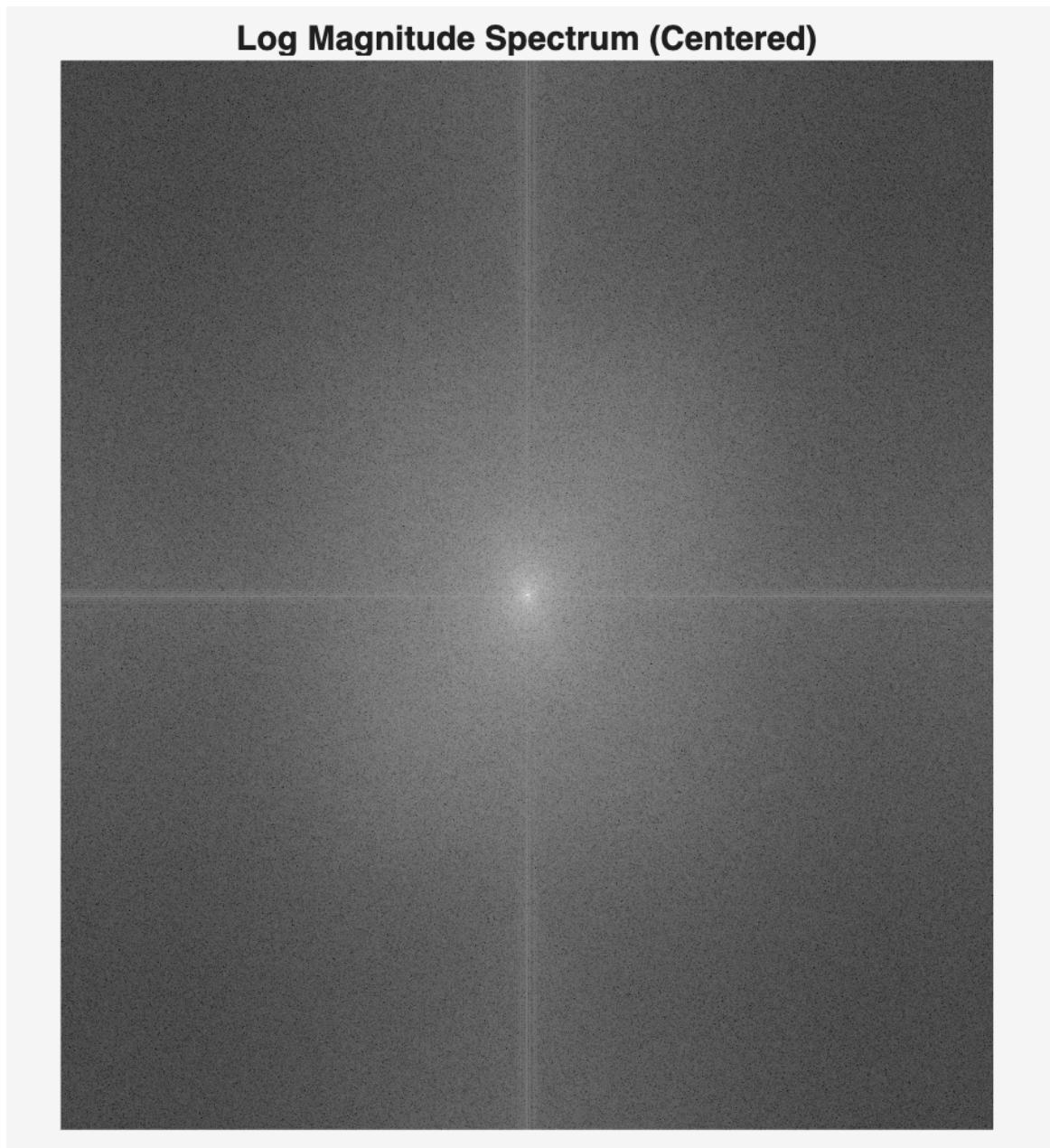


Figure 2: Preperation 2

Preparation 3: Displaying k-space Magnitude Spectrum

Objective

To correctly visualize the Fourier-domain (k-space) representation of an image.

Theory

The 2D Fourier transform of an image produces complex-valued k-space data:

$$D(k_x, k_y) = \mathcal{F}\{m(x, y)\}$$

In MRI, the DC component (center of k-space) has a magnitude that is significantly larger than other frequency components. Direct visualization of $|D|$ leads to poor contrast because the

dynamic range is extremely large.

To compress this dynamic range, a logarithmic transformation is applied:

$$\text{Display} = \log(|D| + 1)$$

The addition of 1 prevents numerical issues due to:

$$\log(0)$$

This transformation allows high-frequency components to become visible while preventing the DC component from dominating the display.

MATLAB Implementation

To display centered k-space magnitude spectrum:

```
imshow(log(abs(d) + 1), [])
```

Where:

- d = centered FFT of the image
- $\text{abs}(d)$ = magnitude of complex k-space
- $\log()$ = dynamic range compression
- $[]$ = automatic intensity scaling

This visualization method was used throughout the homework when displaying k-space data.



Figure 3: Preperation 3

Preparation 4: Quantitative Image Quality Assessment (PSNR & SSIM)

Objective

To quantitatively evaluate reconstruction quality using objective image similarity metrics.

Theory

Two standard image quality assessment (IQA) metrics were used:

1. Peak Signal-to-Noise Ratio (PSNR)

PSNR measures the logarithmic ratio between the maximum possible pixel intensity and the mean squared error (MSE) between two images.

$$\text{MSE} = \frac{1}{N_x N_y} \sum_{x,y} (m_{\text{ref}}(x,y) - m(x,y))^2$$
$$\text{PSNR} = 10 \log_{10} \left(\frac{MAX^2}{\text{MSE}} \right)$$

Where:

- m_{ref} = reference image
- m = reconstructed image
- MAX = maximum pixel intensity value

Higher PSNR indicates smaller reconstruction error.

2. Structural Similarity Index (SSIM)

SSIM measures structural similarity between two images by comparing:

- Luminance
- Contrast
- Structural information

SSIM ranges between:

$$0 \leq \text{SSIM} \leq 1$$

- $\text{SSIM} \approx 1 \rightarrow$ highly similar images
- $\text{SSIM} \approx 0 \rightarrow$ low structural similarity

Unlike PSNR, SSIM better correlates with human visual perception.

Important Normalization Step

Before computing PSNR and SSIM, both images must be normalized to the same intensity scale:

```
max_val = max(m_ref(:));  
  
m_ref_norm = m_ref / max_val;  
m_test_norm = m_test / max_val;
```

MATLAB Implementation

```
psnr_val = psnr(m_test_norm, m_ref_norm);  
ssim_val = ssim(m_test_norm, m_ref_norm);
```

These metrics were used throughout Parts 2–4 to quantitatively compare reconstruction methods.



Figure 4: Preparation 4

Preparation 5: GE Phantom Data

Objective

To load and inspect the provided MRI k-space dataset and prepare it for reconstruction experiments.

Description of the Data

The file `ge_phantom.mat` contains fully sampled, line-by-line acquired k-space data of a GE phantom.

After loading the file, the following variable was obtained:

- kdata → Complex-valued 2D k-space data
- Size: **256 × 256**
- Type: **double (complex)**

Each column corresponds to one phase-encoding (PE) line.

Reference Image Reconstruction

To obtain the fully sampled reference image m_{ref} , a centered inverse Fourier transform was applied:

$$m(x, y) = \mathcal{F}^{-1}\{D(k_x, k_y)\}$$

where $D(k_x, k_y)$ is the centered k-space data.

The magnitude of the reconstructed image was used as the reference image throughout the homework:

$$m_{\text{ref}}(x, y) = |m(x, y)|$$

This image served as the ground truth for all quantitative comparisons (PSNR and SSIM).

MATLAB Implementation

```
load('ge_phantom.mat')
whos
```

PART 1 - IMAGE DOMAIN AND FOURIER DOMAIN FOR PHOTOS

1.1 Image Representation in Spatial and Fourier Domains

Objective

The objective of this section is to load the given photograph, convert it to grayscale format, define it as the reference image, and compute its centered 2D Fourier transform to examine its frequency-domain representation.

Methodology

The image file image.jpeg was loaded into MATLAB using `imread`. Since the image was stored in RGB format, it was converted to grayscale using `rgb2gray`.

The grayscale image was then converted to double precision using `double()` to ensure compatibility with Fourier transform operations and numerical processing.

The resulting image was defined as the reference image:

$$m_{\text{ref}}(x, y)$$

The spatial dimensions of the image were recorded as:

$$N_x = 1263, \quad N_y = 1100$$

After defining the reference image, its centered 2D Fourier transform was computed using the previously implemented `fft2c` function:

$$D(k_x, k_y) = \mathcal{F}\{m_{\text{ref}}(x, y)\}$$

To visualize the frequency-domain representation, the logarithm of the magnitude spectrum was displayed:

$$\log(|D(k_x, k_y)| + 1)$$

The logarithmic scaling was used to compress the dynamic range of the Fourier coefficients and allow better visualization of high-frequency components.

MATLAB

- Code

```
im = imread('image.jpeg');
im = rgb2gray(im);
size(im)

im = double(im);
class(im)

m_ref = im;
[Nx, Ny] = size(m_ref);
Nx
Ny

% Fourier transform
d = fft2c(m_ref);

% ---- Single Figure ----
figure

subplot(1,2,1)
imshow(m_ref,[])
title('Reference Image (m\_ref)')

subplot(1,2,2)
imshow(log(abs(d)+1),[])
title('Centered Log Magnitude Spectrum')
```

- Output

```
ans =  
  
    1263    1100  
  
ans =  
  
    'double'  
  
Nx =  
  
    1263  
  
Ny =  
  
    1100
```

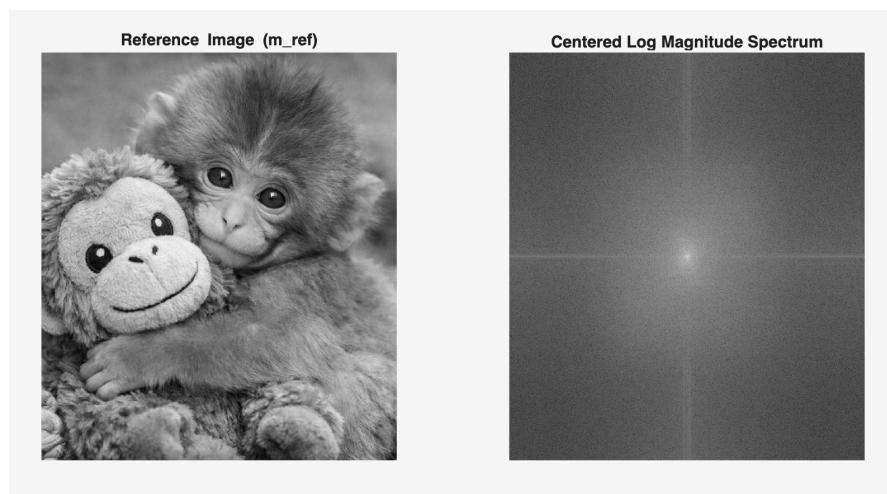


Figure 5: Question 1.1 Figures

1.2 Downsampling by a Factor of 2 and Its Fourier-Domain Effect

Objective

The objective of this section is to investigate the effect of spatial downsampling by a factor of 2 on both the image domain and the Fourier domain representation.

Methodology

The reference image $m_{\text{ref}}(x, y)$ was downsampled by retaining every second pixel in both horizontal and vertical directions:

$$m_{\text{ds2}}(x, y) = m_{\text{ref}}(2x, 2y)$$

This was implemented in MATLAB by selecting:

- Every second row
- Every second column

The resulting image size became:

$$632 \times 550$$

After downsampling, the centered 2D Fourier transform of the downsampled image was computed using `fft2c`.

The log-magnitude spectrum was displayed to observe the impact of downsampling on the frequency-domain structure.

Observations

- The spatial resolution decreases due to removal of half the samples in both directions.
- In the Fourier domain, downsampling leads to **spectral replication (aliasing)** because reducing sampling density increases the spacing between frequency samples.
- High-frequency components overlap, producing visible folding effects in the spectrum.

MATLAB

- Code

```
m_ds2 = m_ref(1:2:end, 1:2:end);
size(m_ds2)

d_ds2 = fft2c(m_ds2);

% ---- Single Figure ----
figure

subplot(1,2,1)
imshow(m_ds2,[])
title('Downsampled Image (Factor 2)')

subplot(1,2,2)
imshow(log(abs(d_ds2)+1),[])
title('Spectrum (Factor 2)')
```

- Output

ans =

632 550

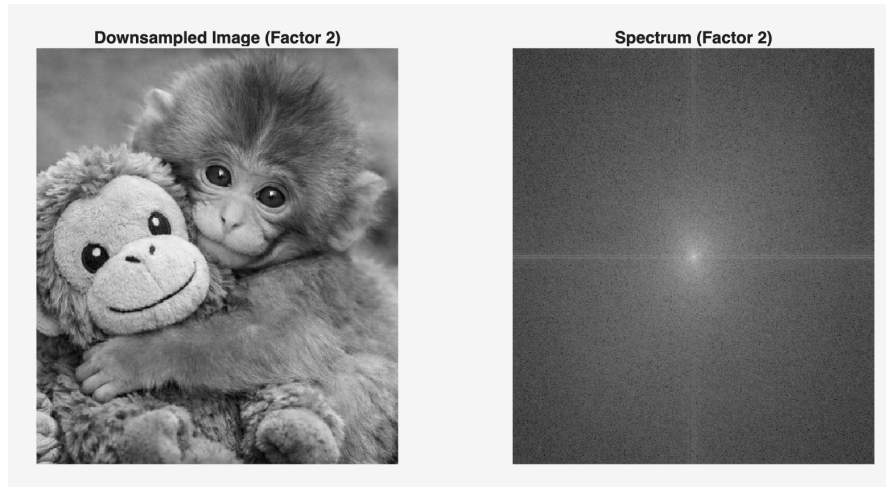


Figure 6: Question 1.2 Figures

1.3 Interpolation via Zero-Padding in the Fourier Domain

Objective

The objective of this section is to reconstruct an image with the original spatial dimensions from the downsampled image by zero-padding its Fourier spectrum and to evaluate the reconstruction quality.

Methodology

The downsampled image m_{ds2} has dimensions:

$$632 \times 550$$

while the original reference image has dimensions:

$$1263 \times 1100$$

To perform interpolation, the following steps were applied:

1. The centered Fourier transform of the downsampled image was computed:

$$D_{ds2}(k_x, k_y) = \mathcal{F}\{m_{ds2}(x, y)\}$$

2. A zero-filled Fourier matrix of size 1263×1100 was created.
3. The spectrum of the downsampled image was placed at the center of the larger zero matrix.
4. A centered inverse Fourier transform was applied to obtain the interpolated image:

$$m_{interp}(x, y) = \mathcal{F}^{-1}\{D_{padded}(k_x, k_y)\}$$

5. The magnitude of the reconstructed image was taken.

Error Analysis

To quantitatively evaluate interpolation quality, the reconstructed image was normalized and compared to the reference image.

The error image was computed as:

$$\text{Error}(x, y) = |m_{\text{interp,norm}} - m_{\text{ref,norm}}|$$

The following quantitative results were obtained:

- **PSNR = 7.8694 dB**
- **SSIM = 0.3596**

These low values indicate that simple zero-padding interpolation does not recover the high-frequency information lost during downsampling. The reconstruction appears blurred and lacks fine details.

MATLAB

- Code

```
size(m_ref)
size(m_ds2)

d_ds2 = fft2c(m_ds2);

d_padded = zeros(Nx, Ny);

row_start = floor((Nx - size(m_ds2,1))/2) + 1;
col_start = floor((Ny - size(m_ds2,2))/2) + 1;

row_end = row_start + size(m_ds2,1) - 1;
col_end = col_start + size(m_ds2,2) - 1;

d_padded(row_start:row_end, col_start:col_end) = d_ds2;

m_interp = ifft2c(d_padded);
m_interp = abs(m_interp);

figure
imshow(m_interp,[])
title('Interpolated Image (Zero-Padded Spectrum)')

max_val = max(m_ref(:));

m_ref_norm = m_ref / max_val;
m_interp_norm = m_interp / max_val;
```

```

error_interp = abs(m_interp_norm - m_ref_norm);

figure
imshow(error_interp,[0 0.2])
title('Error Image (Interpolation)')

psnr_interp = psnr(m_interp_norm, m_ref_norm);
ssim_interp = ssim(m_interp_norm, m_ref_norm);

psnr_interp
ssim_interp

```

- Output

```

ans =

    1263    1100

ans =

    632    550

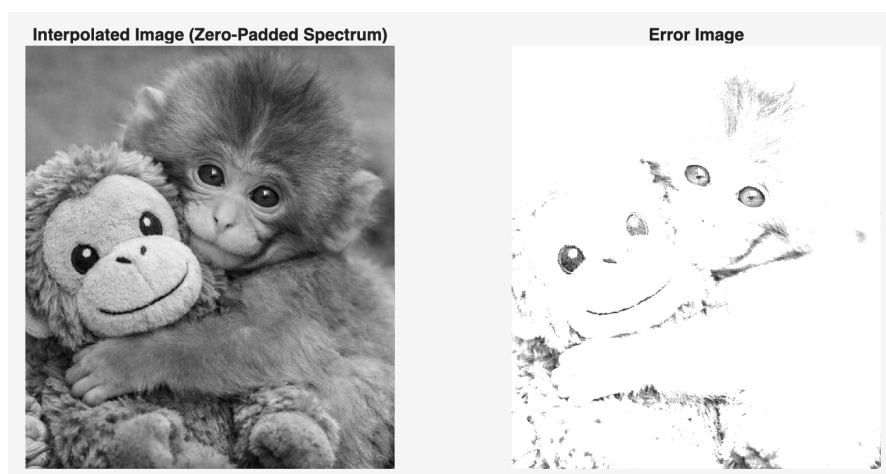
psnr_interp =

    7.8694

ssim_interp =

    0.3596

```



1.4 Downsampling by a Factor of 4 and Fourier-Domain Interpolation

Objective

The objective of this section is to investigate the effects of stronger spatial downsampling (factor of 4) on image quality and evaluate reconstruction performance using Fourier-domain zero-padding interpolation.

Methodology

The reference image $m_{\text{ref}}(x, y)$ was downsampled by retaining every fourth pixel in both spatial directions:

$$m_{\text{ds4}}(x, y) = m_{\text{ref}}(4x, 4y)$$

This resulted in a reduced image size of:

$$316 \times 275$$

The centered 2D Fourier transform of the downsampled image was computed:

$$D_{\text{ds4}}(k_x, k_y) = \mathcal{F}\{m_{\text{ds4}}(x, y)\}$$

To interpolate back to the original image size, the spectrum was zero-padded to match the original dimensions 1263×1100 . The padded spectrum was centered within a larger zero matrix, and the centered inverse Fourier transform was applied:

$$m_{\text{interp4}}(x, y) = \mathcal{F}^{-1}\{D_{\text{padded4}}(k_x, k_y)\}$$

The magnitude of the reconstructed image was used for comparison.

Error Analysis

The interpolated image was normalized using the same maximum intensity value as the reference image.

The reconstruction error was computed as:

$$\text{Error}(x, y) = |m_{\text{interp4, norm}} - m_{\text{ref, norm}}|$$

The following quantitative results were obtained:

- **PSNR = 5.9320 dB**
- **SSIM = 0.0820**

These values are significantly lower than those obtained for factor-2 downsampling, indicating severe information loss. The reconstructed image appears highly blurred, with strong loss of fine structural details due to aggressive downsampling and aliasing.

MATLAB

- Code

```
m_ds4 = m_ref(1:4:end, 1:4:end);
size(m_ds4)

figure
imshow(m_ds4,[])
title('Downsampled Image (Factor 4)')

d_ds4 = fft2c(m_ds4);

figure
imshow(log(abs(d_ds4)+1),[])
title('Spectrum of Downsampled Image (Factor 4)')

d_padded4 = zeros(Nx, Ny);

row_start4 = floor((Nx - size(m_ds4,1))/2) + 1;
col_start4 = floor((Ny - size(m_ds4,2))/2) + 1;

row_end4 = row_start4 + size(m_ds4,1) - 1;
col_end4 = col_start4 + size(m_ds4,2) - 1;

d_padded4(row_start4:row_end4, col_start4:col_end4) = d_ds4;

m_interp4 = ifft2c(d_padded4);
m_interp4 = abs(m_interp4);

figure
imshow(m_interp4,[])
title('Interpolated Image (Factor 4)')

m_interp4_norm = m_interp4 / max_val;

error_interp4 = abs(m_interp4_norm - m_ref_norm);

figure
imshow(error_interp4,[0 0.2])
title('Error Image (Interpolation Factor 4)')

psnr_interp4 = psnr(m_interp4_norm, m_ref_norm);
ssim_interp4 = ssim(m_interp4_norm, m_ref_norm);

psnr_interp4
ssim_interp4
```

- Output

```
ans =  
  
    316    275  
  
psnr_interp4 =  
  
    5.9320  
  
ssim_interp4 =  
  
    0.0820
```

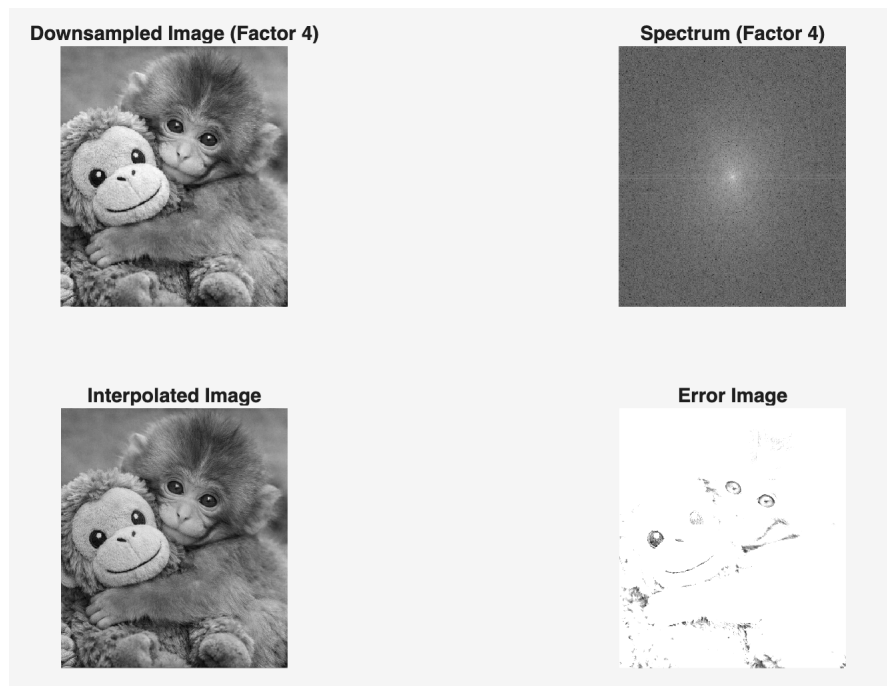


Figure 8: Question 1.4 Figures

1.5 Evaluation of PSNR and SSIM

Discussion

In Part 1, two levels of downsampling (factor 2 and factor 4) were applied, followed by Fourier-domain zero-padding interpolation.

The quantitative results were →

Visually, the following observations were made:

Downsampling Factor	PSNR (dB)	SSIM
2	7.8694	0.35
4	5.9320	0.08

- The interpolated image for factor 2 appeared noticeably blurred but still retained coarse structural content.
- The interpolated image for factor 4 exhibited severe degradation, strong blurring, and substantial loss of fine details.

Table 1: 1.5 Comparison

The quantitative metrics reflect these observations:

- **PSNR decreases** as the downsampling factor increases, indicating larger reconstruction error.
- **SSIM decreases significantly**, particularly for factor 4, reflecting major structural distortion.

SSIM shows a stronger drop compared to PSNR, which aligns well with visual perception because SSIM measures structural similarity rather than only pixel-wise error.

PART 2 - EFFECTS OF PHASE ENCODING LINE MANIPULATION

2.1 Reconstruction from Fully Sampled k-Space Data

Objective

The objective of this section is to reconstruct the MRI image from the fully sampled k-space dataset and examine its magnitude, phase, real, and imaginary components.

Methodology

The file `ge_phantom.mat` was loaded into MATLAB, which provided the complex-valued k-space dataset:

$$kdata \in \mathbb{C}^{256 \times 256}$$

The magnitude spectrum of the full k-space data was displayed using logarithmic scaling to visualize frequency distribution.

The centered inverse Fourier transform was then applied:

$$m(x, y) = \mathcal{F}^{-1}\{kdata(k_x, k_y)\}$$

The magnitude of the reconstructed image was defined as the reference MRI image:

$$m_{\text{ref}}(x, y) = |m(x, y)|$$

Additionally, the following components were visualized:

- Phase image: $\angle m(x, y)$
- Real part: $\Re\{m(x, y)\}$
- Imaginary part: $\Im\{m(x, y)\}$

The maximum magnitude value of the reference image was stored for normalization in later PSNR and SSIM calculations.

Observations

- The k-space magnitude spectrum shows strong energy concentration near the center (low frequencies).
- The magnitude image provides the anatomical structure of the phantom.
- The phase image contains spatial phase variations that are not visible in the magnitude image.
- The real and imaginary components individually do not resemble the final magnitude image but together form the complex-valued reconstruction.

MATLAB

- Script

```
load('ge_phantom.mat')
whos

m = ifft2c(kdata);
m_ref = abs(m);

max_val = max(m_ref(:));

% ---- Single Figure (2x3 Layout) ----
figure

subplot(2,3,1)
imshow(log(abs(kdata)+1),[])
title('Full k-space Spectrum')

subplot(2,3,2)
imshow(m_ref,[])
title('Magnitude Image (Reference)')

subplot(2,3,3)
imshow(angle(m),[])
title('Phase Image')

subplot(2,3,4)
imshow(real(m),[])
title('Real Part')

subplot(2,3,5)
imshow(imag(m),[])
title('Imaginary Part')
```

- Output

Name	Size	Bytes	Class
Attributes			
Nx	1x1	8	double
Ny	1x1	8	double
ans	1x2	16	double
ax1	1x1	8	matlab.gr
aphics.axis.Axes			
ax2	1x1	8	matlab.gr
aphics.axis.Axes			
col_end	1x1	8	double
col_end4	1x1	8	double
col_start	1x1	8	double
col_start4	1x1	8	double
d	1263x1100	22228800	double
complex			
d_ds2	632x550	5561600	double
complex			
d_ds4	316x275	1390400	double
complex			
d_padded	1263x1100	22228800	double
complex			
d_padded4	1263x1100	22228800	double
complex			
error_central	256x256	524288	double
error_central_quarter	256x256	524288	double
error_interp	1263x1100	11114400	double
error_interp4	1263x1100	11114400	double
error_iter	256x256	524288	double
error_k_mag	256x256	524288	double
error_pc	256x256	524288	double
error_pc_16	256x256	524288	double
error_pc_32	256x256	524288	double
error_pc_64	256x256	524288	double
error_pc_8	256x256	524288	double
error_zero	256x256	524288	double
error_zero4	256x256	524288	double
error_zf	256x256	524288	double
error_zf_9_16	256x256	524288	double
f1	1x1	8	matlab.u
i.Figure			
f2	1x1	8	matlab.u
i.Figure			
fig1	1x1	8	matlab.u
i.Figure			

fig2	1x1	8	matlab.u
i.Figure			
i	1x1	8	double
im	1263x1100	11114400	double
k_17_32	256x256	1048576	double
complex			
k_33_64	256x256	1048576	double
complex			
k_5_8	256x256	1048576	double
complex			
k_9_16	256x256	1048576	double
complex			
k_central	256x128	524288	double
complex			
k_central_quarter	256x256	1048576	double
complex			
k_central_zero	256x256	1048576	double
complex			
k_iter	256x256	1048576	double
complex			
k_mag	256x256	524288	double
k_phase	256x256	1048576	double
complex			
k_phase_16	256x256	1048576	double
complex			
k_phase_32	256x256	1048576	double
complex			
k_phase_64	256x256	1048576	double
complex			
k_phase_8	256x256	1048576	double
complex			
k_phase_pocs	256x256	1048576	double
complex			
k_skip	256x128	524288	double
complex			
k_zero	256x256	1048576	double
complex			
k_zero4	256x256	1048576	double
complex			
kdata	256x256	1048576	double
complex			
m	256x256	1048576	double
complex			
m_central	256x128	524288	double
complex			
m_central_quarter	256x256	1048576	double
complex			

m_central_quarter_mag	256x256	524288	double
m_central_quarter_norm	256x256	524288	double
m_central_zero	256x256	1048576	double
complex			
m_central_zero_mag	256x256	524288	double
m_central_zero_norm	256x256	524288	double
m_ds2	632x550	2780800	double
m_ds4	316x275	695200	double
m_from_k_mag	256x256	1048576	double
complex			
m_from_k_mag_mag	256x256	524288	double
m_from_k_mag_norm	256x256	524288	double
m_interp	1263x1100	11114400	double
m_interp4	1263x1100	11114400	double
m_interp4_norm	1263x1100	11114400	double
m_interp_norm	1263x1100	11114400	double
m_iter	256x256	1048576	double
complex			
m_lowres_16	256x256	1048576	double
complex			
m_lowres_32	256x256	1048576	double
complex			
m_lowres_64	256x256	1048576	double
complex			
m_lowres_8	256x256	1048576	double
complex			
m_mag_only	256x256	524288	double
m_output	256x256	524288	double
m_output_norm	256x256	524288	double
m_pc	256x256	524288	double
m_pc_16	256x256	524288	double
m_pc_16_norm	256x256	524288	double
m_pc_32	256x256	524288	double
m_pc_32_norm	256x256	524288	double
m_pc_64	256x256	524288	double
m_pc_64_norm	256x256	524288	double
m_pc_8	256x256	524288	double
m_pc_8_norm	256x256	524288	double
m_pc_norm	256x256	524288	double
m_phase_lowres	256x256	1048576	double
complex			
m_phase_pocs	256x256	1048576	double
complex			
m_ref	1263x1100	11114400	double
m_ref_norm	1263x1100	11114400	double
m_skip	256x128	524288	double
complex			

m_zero	256x256	1048576	double
complex			
m_zero4	256x256	1048576	double
complex			
m_zero4_mag	256x256	524288	double
m_zero4_norm	256x256	524288	double
m_zero_mag	256x256	524288	double
m_zero_norm	256x256	524288	double
m_zf	256x256	1048576	double
complex			
m_zf_17_32	256x256	1048576	double
complex			
m_zf_17_32_mag	256x256	524288	double
m_zf_17_32_norm	256x256	524288	double
m_zf_33_64	256x256	1048576	double
complex			
m_zf_33_64_mag	256x256	524288	double
m_zf_33_64_norm	256x256	524288	double
m_zf_9_16	256x256	1048576	double
complex			
m_zf_9_16_mag	256x256	524288	double
m_zf_9_16_norm	256x256	524288	double
m_zf_mag	256x256	524288	double
m_zf_norm	256x256	524288	double
max_val	1x1	8	double
p	256x256	1048576	double
complex			
p_16	256x256	1048576	double
complex			
p_32	256x256	1048576	double
complex			
p_64	256x256	1048576	double
complex			
p_8	256x256	1048576	double
complex			
p_pocs	256x256	1048576	double
complex			
phase_lowres	256x256	524288	double
phase_lowres_16	256x256	524288	double
phase_lowres_32	256x256	524288	double
phase_lowres_64	256x256	524288	double
phase_lowres_8	256x256	524288	double
phase_pocs	256x256	524288	double
psnr_central	1x1	8	double
psnr_central_quarter	1x1	8	double
psnr_interp	1x1	8	double
psnr_interp4	1x1	8	double

psnr_iter	1x1	8	double
psnr_k_mag	1x1	8	double
psnr_pc	1x1	8	double
psnr_pc_16	1x1	8	double
psnr_pc_32	1x1	8	double
psnr_pc_64	1x1	8	double
psnr_pc_8	1x1	8	double
psnr_zero	1x1	8	double
psnr_zero4	1x1	8	double
psnr_zf	1x1	8	double
psnr_zf_17_32	1x1	8	double
psnr_zf_33_64	1x1	8	double
psnr_zf_9_16	1x1	8	double
row_end	1x1	8	double
row_end4	1x1	8	double
row_start	1x1	8	double
row_start4	1x1	8	double
ssim_central	1x1	8	double
ssim_central_quarter	1x1	8	double
ssim_interp	1x1	8	double
ssim_interp4	1x1	8	double
ssim_iter	1x1	8	double
ssim_k_mag	1x1	8	double
ssim_pc	1x1	8	double
ssim_pc_16	1x1	8	double
ssim_pc_32	1x1	8	double
ssim_pc_64	1x1	8	double
ssim_pc_8	1x1	8	double
ssim_zero	1x1	8	double
ssim_zero4	1x1	8	double
ssim_zf	1x1	8	double
ssim_zf_17_32	1x1	8	double
ssim_zf_33_64	1x1	8	double
ssim_zf_9_16	1x1	8	double

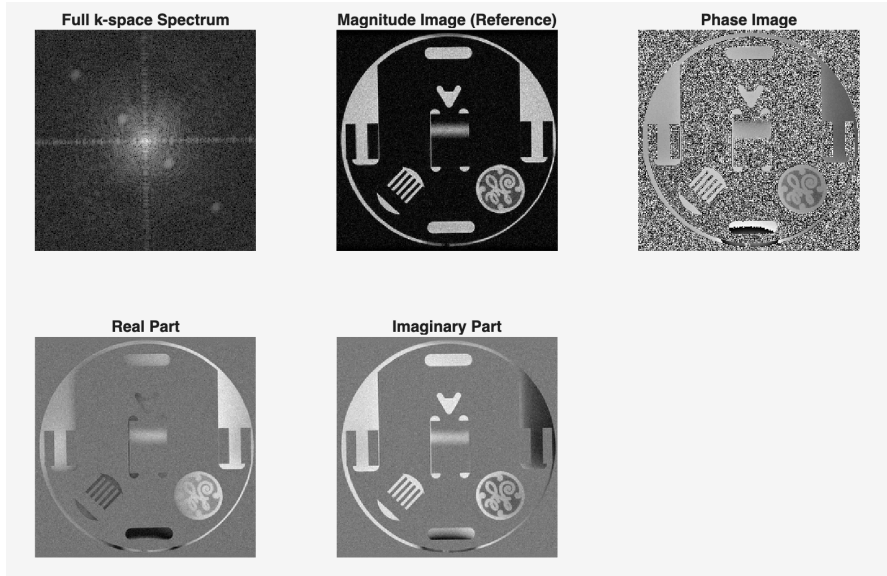


Figure 9: Question 2.1 Figures

2.2 Reconstruction After Skipping Every Second Phase-Encoding Line

Objective

The objective of this section is to investigate the effect of skipping every second phase-encoding (PE) line in k-space and reconstructing the image using only the remaining acquired lines.

Methodology

To simulate undersampling in the phase-encoding direction, every second column of the k-space data was retained:

$$k_{\text{skip}} = kdata(:, 1 : 2 : end)$$

This reduced the k-space size from:

$$256 \times 256$$

to:

$$256 \times 128$$

A centered inverse Fourier transform was applied to reconstruct the image from this reduced k-space dataset:

$$m_{\text{skip}}(x, y) = \mathcal{F}^{-1}\{k_{\text{skip}}(k_x, k_y)\}$$

The magnitude of the reconstructed image was displayed.

Observations

- The reconstructed image has reduced resolution in the phase-encoding direction.

- The image appears compressed in one spatial dimension due to reduced sampling.
- The matrix size of the reconstructed image becomes 256 $\times$ 128, reflecting the reduced k-space dimension.
- No zero-filling was applied in this step; the reconstruction directly used the truncated k-space matrix.

MATLAB

- Code

```
k_skip = kdata(:,1:2:end);
size(k_skip)

m_skip = ifft2c(k_skip);

figure
imshow(abs(m_skip),[])
title('Image from Skipped PE Lines')
```

- Output

```
ans =
    256    128
```

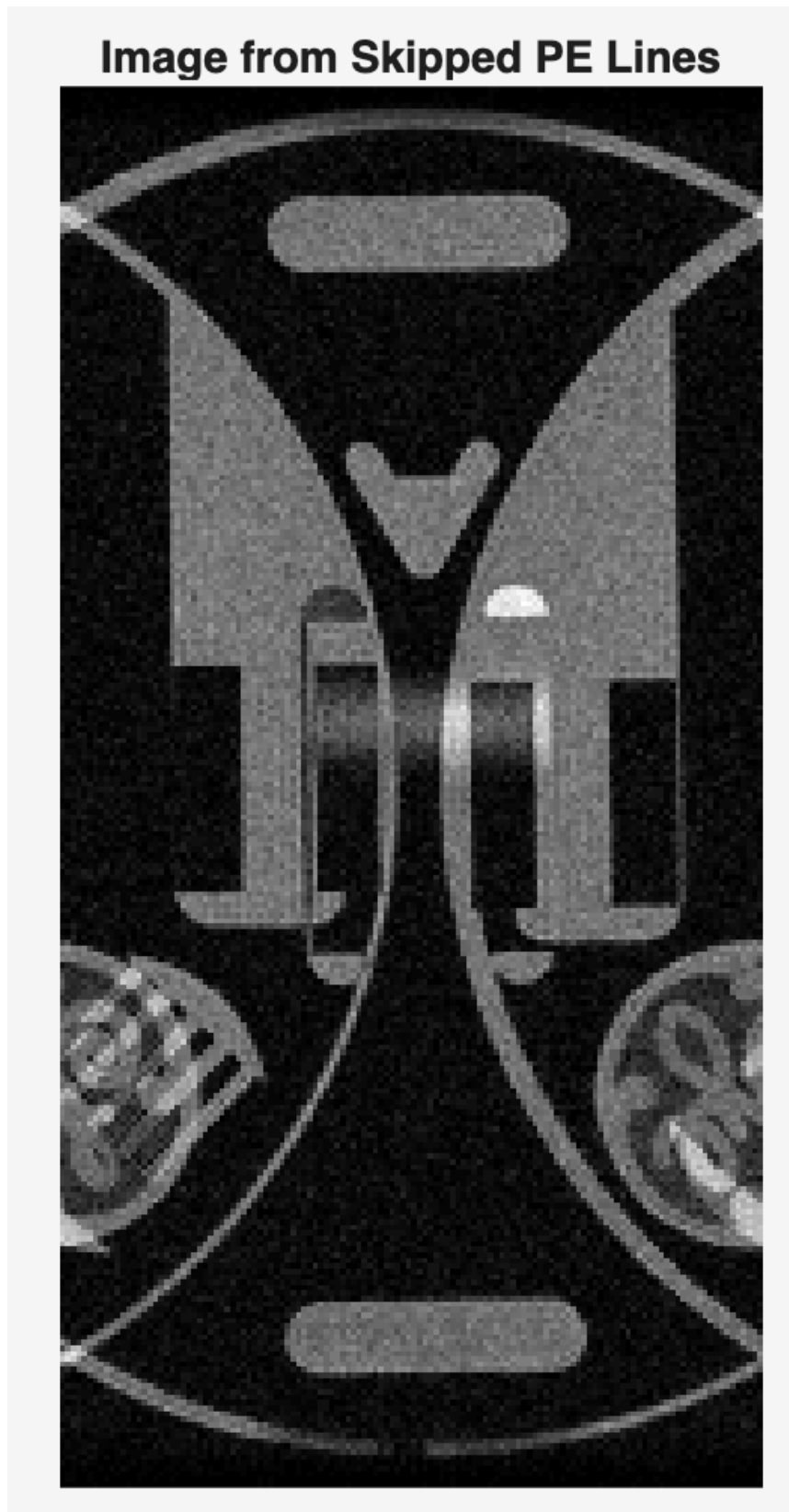


Figure 10: Question 2.2 Figure

2.3 Reconstruction After Zeroing Every Second Phase-Encoding Line

Objective

The objective of this section is to investigate the effect of undersampling by zeroing every second phase-encoding (PE) line while maintaining the original k-space matrix size.

Methodology

Instead of removing columns (as in 2.2), every second PE line was set to zero while preserving the full matrix size:

$$k_{\text{zero}}(:, 2 : 2 : \text{end}) = 0$$

This results in a 256×256 k-space matrix where half of the phase-encoding lines contain zero values.

A centered inverse Fourier transform was applied:

$$m_{\text{zero}}(x, y) = \mathcal{F}^{-1}\{k_{\text{zero}}(k_x, k_y)\}$$

The magnitude of the reconstructed image was displayed and compared with the reference image.

To quantify reconstruction quality, the images were normalized using the maximum value of the reference image and the error image was computed:

$$\text{Error}(x, y) = |m_{\text{zero, norm}} - m_{\text{ref, norm}}|$$

Quantitative Results

- **PSNR = 14.1878 dB**
- **SSIM = 0.4799**

Observations

- Unlike Part 2.2, the reconstructed image retains the original spatial dimensions 256×256 .
- The image exhibits visible aliasing artifacts due to missing phase-encoding lines.
- Compared to direct skipping (2.2), zero-filling preserves matrix size but does not prevent aliasing.
- The moderate PSNR and SSIM values reflect noticeable structural degradation compared to the fully sampled reference.

MATLAB

- **Code**

```
k_zero = kdata;  
k_zero(:, 2:2:end) = 0;  
size(k_zero)  
  
m_zero = ifft2c(k_zero);
```

```

m_zero_mag = abs(m_zero);
m_zero_norm = m_zero_mag / max_val;
m_ref_norm = m_ref / max_val;

error_zero = abs(m_zero_norm - m_ref_norm);

% ---- Single Figure ----
figure

subplot(1,2,1)
imshow(m_zero_mag,[])
title('Reconstruction (Zeroed PE Lines)')

subplot(1,2,2)
imshow(error_zero,[0 0.2])
title('Error Image')

% Metrics
psnr_zero = psnr(m_zero_norm, m_ref_norm);
ssim_zero = ssim(m_zero_norm, m_ref_norm);

psnr_zero
ssim_zero

```

- Output

```

ans =

    256    256

psnr_zero =

    14.1878

ssim_zero =

    0.4799

```

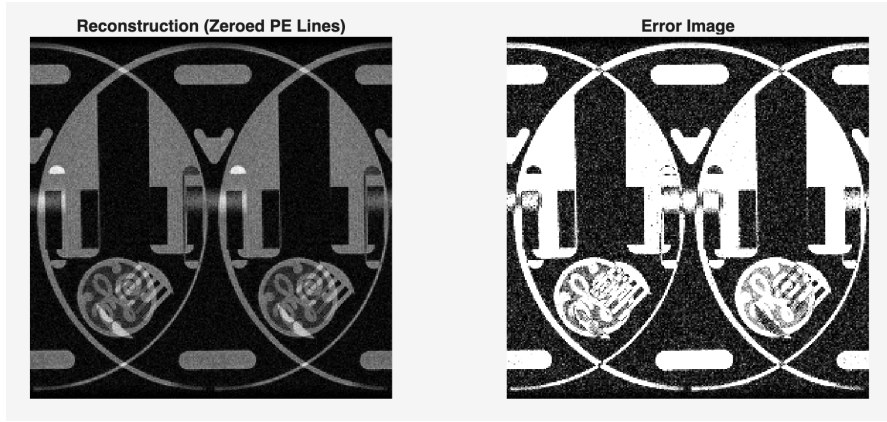


Figure 11: Question 2.3 Figures

2.4 Reconstruction Using Only 1/4 of the Phase-Encoding Lines (Zero-Filled)

Objective

The objective of this section is to investigate the effect of more aggressive undersampling in the phase-encoding (PE) direction by retaining only one out of every four PE lines while zeroing the remaining lines.

Methodology

To simulate stronger undersampling, only every fourth PE line was preserved in k-space, and the remaining three lines were set to zero:

$$k_{\text{zero4}}(:, 2 : 4 : \text{end}) = 0$$

$$k_{\text{zero4}}(:, 3 : 4 : \text{end}) = 0$$

$$k_{\text{zero4}}(:, 4 : 4 : \text{end}) = 0$$

This preserves only 25% of the phase-encoding data while keeping the overall k-space matrix size at 256×256 .

A centered inverse Fourier transform was then applied:

$$m_{\text{zero4}}(x, y) = \mathcal{F}^{-1}\{k_{\text{zero4}}(k_x, k_y)\}$$

The magnitude of the reconstructed image was compared to the fully sampled reference image.

The error image was computed after normalization, and quantitative metrics were evaluated using PSNR and SSIM.

Quantitative Results

- PSNR = 12.5827 dB
- SSIM = 0.2481

Discussion

Compared to the case where half of the PE lines were retained (Section 2.3), the image quality degrades further when only 1/4 of the lines are preserved.

Visually, the reconstructed image exhibits:

- Strong aliasing artifacts
- Significant loss of structural detail
- Increased blurring

The decrease in both PSNR and SSIM reflects the severe structural distortion caused by aggressive undersampling.

Notably, although the matrix size remains unchanged due to zero-filling, missing phase-encoding information leads to substantial image degradation.

MATLAB

- Code

```
k_zero4 = kdata;
k_zero4(:,2:4:end) = 0;
k_zero4(:,3:4:end) = 0;
k_zero4(:,4:4:end) = 0;

size(k_zero4)

m_zero4 = ifft2c(k_zero4);
m_zero4_mag = abs(m_zero4);

m_zero4_norm = m_zero4_mag / max_val;
m_ref_norm = m_ref / max_val;

error_zero4 = abs(m_zero4_norm - m_ref_norm);

% ---- Single Figure ----
figure

subplot(1,2,1)
imshow(m_zero4_mag,[])
title('Reconstruction (1/4 PE Lines Kept)')

subplot(1,2,2)
imshow(error_zero4,[0 0.2])
title('Error Image')

% Metrics
psnr_zero4 = psnr(m_zero4_norm, m_ref_norm);
ssim_zero4 = ssim(m_zero4_norm, m_ref_norm);
```

```
psnr_zero4  
ssim_zero4
```

- Output

```
ans =  
  
    256    256  
  
psnr_zero4 =  
  
    12.5827  
  
ssim_zero4 =  
  
    0.2481
```

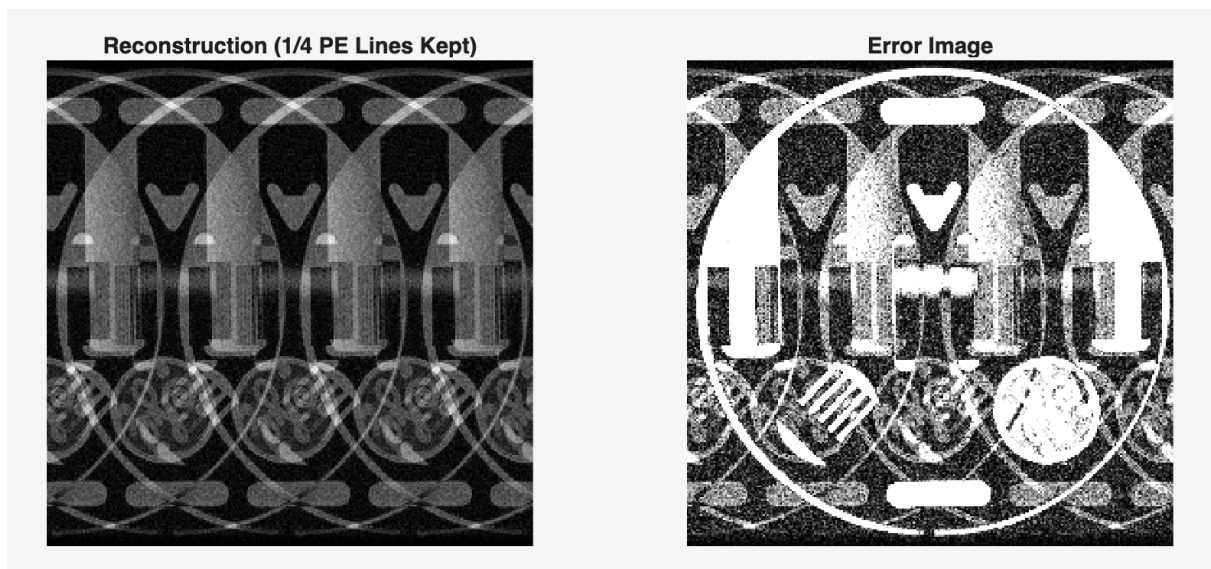


Figure 12: Question 2.4 Figures

2.5 Reconstruction Using Only the Central Half of k-Space (Without Zero-Filling)

Objective

The objective of this section is to investigate the effect of retaining only the central half of k-space in the phase-encoding direction and reconstructing the image without zero-filling.

Methodology

Instead of keeping every other phase-encoding line, only the central portion of k-space was retained. Specifically, columns 65 through 192 were selected:

$$k_{\text{central}} = kdata(:, 65 : 192)$$

This corresponds to keeping the central 50% of the phase-encoding lines. The resulting matrix size becomes:

$$256 \times 128$$

A centered inverse Fourier transform was then applied directly to this reduced k-space matrix:

$$m_{\text{central}}(x, y) = \mathcal{F}^{-1}\{k_{\text{central}}(k_x, k_y)\}$$

The magnitude of the reconstructed image was displayed.

Discussion

- The reconstructed image has dimensions 256×128 , reflecting the reduced k-space size.
- Unlike skipping every other line, this method preserves the **low-frequency components**, which are concentrated near the center of k-space.
- As a result, the image retains overall structural information but appears blurred due to loss of high-frequency details.
- No aliasing is observed in this case, since only central frequencies were retained.

MATLAB

- Code

```
k_central = kdata(:,65:192);  
size(k_central)  
m_central = ifft2c(k_central);  
  
figure  
imshow(abs(m_central),[])  
title('Image from Central Half of k-space')
```

- Output

```
ans =  
    256    128
```

Image from Central Half of k-space

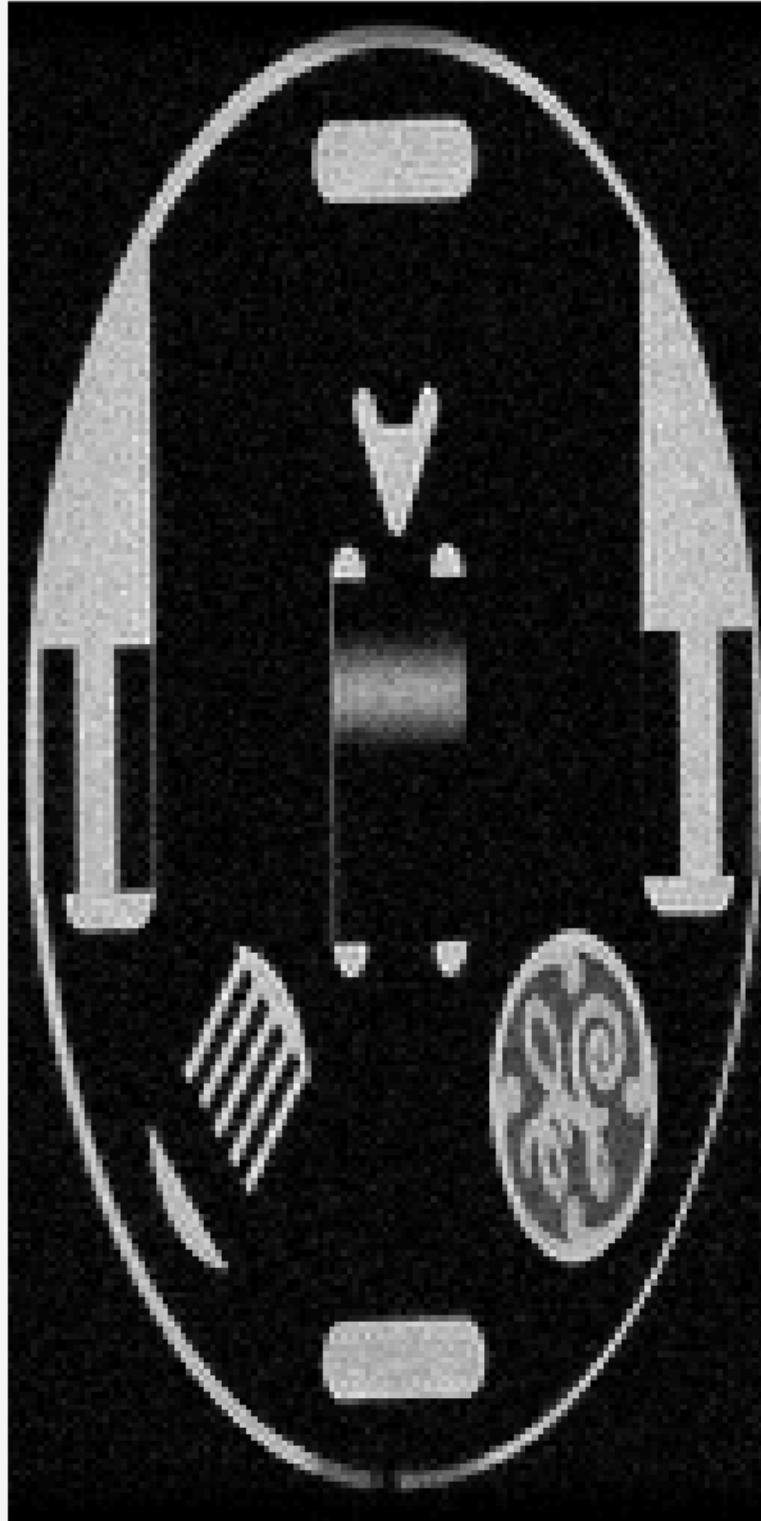


Figure 13: Question 2.5 Figure

2.6 Reconstruction Using Only the Central Half of k-Space (Zero-Filled Outer Region)

Objective

The objective of this section is to reconstruct the image using only the central half of k-space while preserving the original matrix size by zero-filling the outer high-frequency regions.

Methodology

In this case, the full 256 \times 256 k-space matrix was preserved, but the outer phase-encoding lines were set to zero:

$$k_{\text{central_zero}}(:, 1 : 64) = 0$$

$$k_{\text{central_zero}}(:, 193 : 256) = 0$$

This retains only the central 50% of k-space while maintaining the original matrix size.

A centered inverse Fourier transform was applied:

$$m_{\text{central_zero}}(x, y) = \mathcal{F}^{-1}\{k_{\text{central_zero}}(k_x, k_y)\}$$

The magnitude image was compared to the fully sampled reference image using PSNR and SSIM.

Quantitative Results

- **PSNR = 26.9586 dB**
- **SSIM = 0.7348**

Discussion

Unlike skipping alternate phase-encoding lines (Sections 2.2–2.4), retaining only the central k-space region preserves low-frequency information, which carries the main structural content of the image.

Visually:

- The image appears blurred due to missing high-frequency components.
- No strong aliasing artifacts are observed.
- Structural information remains largely intact.

The relatively high PSNR and SSIM values confirm that low-frequency information dominates overall image structure. This case shows significantly better quality compared to uniform undersampling (Section 2.3 and 2.4).

MATLAB

- Code

```
k_central_zero = kdata;  
k_central_zero(:,1:64) = 0;  
k_central_zero(:,193:256) = 0;  
  
size(k_central_zero)
```

```

m_central_zero = ifft2c(k_central_zero);
m_central_zero_mag = abs(m_central_zero);

m_central_zero_norm = m_central_zero_mag / max_val;
m_ref_norm          = m_ref / max_val;

error_central = abs(m_central_zero_norm - m_ref_norm);

% ---- Single Figure ----
figure

subplot(1,2,1)
imshow(m_central_zero_mag,[])
title('Reconstruction (Central Half Kept)')

subplot(1,2,2)
imshow(error_central,[0 0.2])
title('Error Image')

% Metrics
psnr_central = psnr(m_central_zero_norm, m_ref_norm);
ssim_central = ssim(m_central_zero_norm, m_ref_norm);

psnr_central
ssim_central

```

- Output

```

ans =

    256    256

psnr_central =

    26.9586

ssim_central =

    0.7348

```

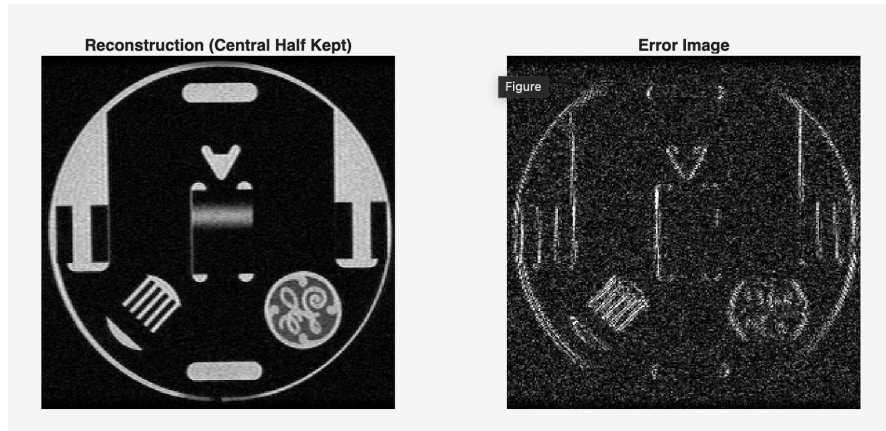


Figure 14: Question 2.6 Figures

2.7 Reconstruction Using Only the Central 1/4 of k-Space (Zero-Filled Outer Region)

Objective

The objective of this section is to analyze the effect of retaining only the central 25% of k-space while zeroing the remaining outer regions and reconstructing the image with preserved matrix size.

Methodology

To retain only the central quarter of k-space, the outer phase-encoding lines were set to zero:

$$k_{\text{central_quarter}}(:, 1 : 96) = 0$$

$$k_{\text{central_quarter}}(:, 161 : 256) = 0$$

This preserves only 25% of the central k-space data while maintaining the full 256×256 matrix size.

A centered inverse Fourier transform was applied:

$$m_{\text{central_quarter}}(x, y) = \mathcal{F}^{-1}\{k_{\text{central_quarter}}(k_x, k_y)\}$$

The magnitude image was then compared to the fully sampled reference image using PSNR and SSIM.

Quantitative Results

- **PSNR = 23.3720 dB**
- **SSIM = 0.5741**

Discussion

Compared to Section 2.6 (central half retained), the reconstruction quality decreases when only the central quarter of k-space is used.

Case	PSNR (dB)	SSIM
Central 1/2	26.9586	0.7348

Visually:

- The image appears noticeably blurred.
- Fine details are significantly lost.
- However, the overall structural content is still recognizable.

Case	PSNR (dB)	SSIM
Central 1/4	23.3720	0.5741

Table 2: Comparison for Question 2.7

The reduction in PSNR and SSIM reflects the increasing loss of frequency information. Since low-frequency components dominate structural information, the image remains recognizable, but quality degradation is clearly measurable.

This confirms that image quality decreases as the retained central k-space ratio decreases.

MATLAB

- Code

```
k_central_quarter = kdata;
k_central_quarter(:,1:96) = 0;
k_central_quarter(:,161:256) = 0;

size(k_central_quarter)
m_central_quarter = ifft2c(k_central_quarter);

figure
imshow(abs(m_central_quarter),[])
title('Image with Central 1/4 of k-space (Zeroed Outer)')
m_central_quarter_mag = abs(m_central_quarter);

m_central_quarter_norm = m_central_quarter_mag / max_val;
m_ref_norm = m_ref / max_val;

error_central_quarter = abs(m_central_quarter_norm - m_ref_norm);

figure
imshow(error_central_quarter,[0 0.2])
title('Error Image (Central 1/4 Zeroed)')
psnr_central_quarter = psnr(m_central_quarter_norm, m_ref_norm);
ssim_central_quarter = ssim(m_central_quarter_norm, m_ref_norm);

psnr_central_quarter
ssim_central_quarter
```

- Output

```

ans =

    256    256

psnr_central_quarter =

    23.3720

ssim_central_quarter =

    0.5741

```

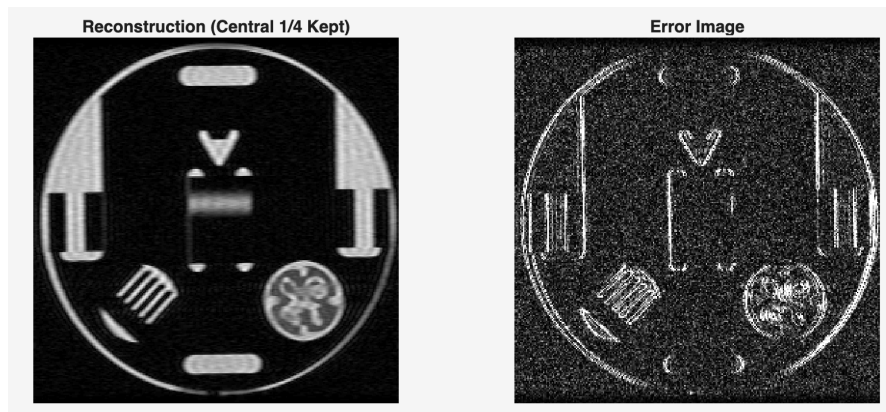


Figure 15: Question 2.7 Figures

2.8 Evaluation of PSNR and SSIM in Part 2

Discussion

In Part 2, several undersampling strategies were applied in the phase-encoding direction:

1. Skipping alternate PE lines
2. Zeroing alternate PE lines
3. Keeping only the central half of k-space
4. Keeping only the central quarter of k-space

The corresponding quantitative results showed clear trends:

- Uniform undersampling (e.g., 1/2 or 1/4 lines retained uniformly) resulted in low PSNR and SSIM values due to strong aliasing artifacts.
- Retaining central k-space data yielded significantly higher PSNR and SSIM values, even when only 50% or 25% of the data was used.
- As the retained central k-space ratio decreased, both PSNR and SSIM gradually decreased.

These numerical trends were consistent with visual observations:

- Uniform undersampling produced severe aliasing artifacts, clearly visible in the reconstructed images.
- Central k-space retention preserved structural information but introduced blurring due to loss of high-frequency components.
- The reduction in PSNR and SSIM correlated well with increasing visual degradation.

However, an important observation is that PSNR alone does not fully reflect perceptual quality differences between aliasing and blurring artifacts. Images with strong aliasing sometimes had PSNR values comparable to moderately blurred images, even though visually they appeared worse.

SSIM, on the other hand, better reflected structural distortions and showed stronger sensitivity to visually noticeable degradation.

Conclusion

Overall, both PSNR and SSIM generally reflected the visual observations in this part of the homework. However, SSIM provided a better indication of perceptual image quality, especially when structural artifacts such as aliasing were present.

PART 3 - PHASE-COMPENSATED MRI IMAGE

3.1 Phase-Compensated Full k-space Image ($\pm 1/8$ Central Data)

In this part, the central $\pm 1/8$ of k-space data (central 25% in PE direction, corresponding to 64 PE lines) was used to estimate a low-resolution phase image.

The goal is to estimate:

$$p(x, y) = e^{j\angle m_{low}(x, y)}$$

where $m_{low}(x, y)$ is obtained by inverse Fourier transforming only the central portion of k-space.

The phase-compensated image is computed as:

$$m_{+2}(x, y) = |\text{Re} \{m(x, y) p^*(x, y)\}|$$

where:

- $m(x, y)$ is the full complex reconstruction
- $p^*(x, y)$ is the complex conjugate of the estimated phase

Procedure

1. The central 64 PE lines were retained.
2. The low-resolution phase image was computed.
3. The phase function was constructed.
4. The full image was phase-compensated.

5. The error image, PSNR, and SSIM were computed.

MATLAB

- Code

```
% Extract central  $\pm 1/8$  k-space (64 PE lines)
k_phase_8 = zeros(size(kdata));
k_phase_8(:,97:160) = kdata(:,97:160);

size(k_phase_8)

ans =

    256    256

% Low-resolution image
m_lowres_8 = ifft2c(k_phase_8);

% Phase image
phase_lowres_8 = angle(m_lowres_8);

figure
imshow(phase_lowres_8,[])
title('Low-Resolution Phase ( $\pm 1/8$ )')

% Phase function
p_8 = exp(1j * phase_lowres_8);

% Phase-compensated full image
m_pc_8 = abs(real(m .* conj(p_8)));

figure
imshow(m_pc_8,[])
title('Phase-Compensated Image ( $\pm 1/8$ )')

% Normalization and error
m_pc_8_norm = m_pc_8 / max_val;
m_ref_norm = m_ref / max_val;

error_pc_8 = abs(m_pc_8_norm - m_ref_norm);

figure
```

```

imshow(error_pc_8,[0 0.2])
title('Error Image (Phase Comp  $\pm 1/8$ )')

% Metrics
psnr_pc_8 = psnr(m_pc_8_norm, m_ref_norm);
ssim_pc_8 = ssim(m_pc_8_norm, m_ref_norm);

psnr_pc_8
ssim_pc_8

```

- Output

```

ans =

    256    256

psnr_pc_8 =

    32.2346

ssim_pc_8 =

    0.878

```

Discussion

The phase-compensated image using $\pm 1/8$ central k-space data shows very strong agreement with the reference image.

- The PSNR value of **32.23 dB** indicates low reconstruction error.
- The SSIM value of **0.8780** indicates high structural similarity.
- Visually, the reconstruction is very close to the original magnitude image.

This confirms that using 25% central k-space data provides a sufficiently accurate phase estimate for high-quality reconstruction.

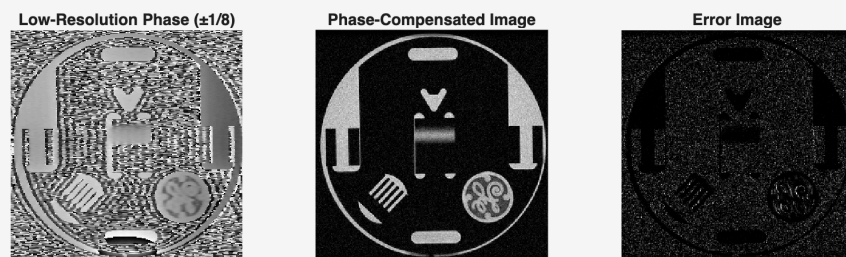


Figure 16: Question 3.1 Figures

3.2 Phase-Compensated Image Using $\pm 1/16$ Central k-Space Data

In this part, the central $\pm 1/16$ of k-space data was used to estimate the low-resolution phase image.

Since the full PE dimension contains 256 lines:

- $\pm 1/8 \rightarrow 64$ PE lines
- $\pm 1/16 \rightarrow$ **32 PE lines**

Thus, only the central 32 PE lines were retained to compute the phase estimate.

The phase image was obtained as:

$$p(x, y) = e^{j\angle m_{low}(x, y)}$$

where $m_{low}(x, y)$ is reconstructed from the central 32 PE lines.

The phase-compensated image was then computed using:

$$m_{+2}(x, y) = |\text{Re} \{m(x, y) p^*(x, y)\}|$$

where:

- $m(x, y)$ is the full complex reconstruction
- $p^*(x, y)$ is the complex conjugate of the estimated phase

MATLAB

- Code

```
% --- Extract central  $\pm 1/16$  (32 PE lines) ---
k_phase_16 = zeros(size(kdata));
k_phase_16(:, 113:144) = kdata(:, 113:144);

size(k_phase_16)

ans =

    256    256

% Low-resolution phase
m_lowres_16 = ifft2c(k_phase_16);
phase_lowres_16 = angle(m_lowres_16);

% Phase function
p_16 = exp(1j * phase_lowres_16);
```

```

% Phase-compensated image
m_pc_16 = abs(real(m .* conj(p_16)));

% Normalization
m_pc_16_norm = m_pc_16 / max_val;
m_ref_norm    = m_ref / max_val;

% Error image
error_pc_16 = abs(m_pc_16_norm - m_ref_norm);

% Metrics
psnr_pc_16 = psnr(m_pc_16_norm, m_ref_norm);
ssim_pc_16 = ssim(m_pc_16_norm, m_ref_norm);

psnr_pc_16
ssim_pc_16

```

- Output

```

psnr_pc_16
ssim_pc_16

psnr_pc_16 =

    31.0503

ssim_pc_16 =

    0.8428

>>

```

Discussion

Compared to Part 3.1 ($\pm 1/8$), reducing the central k-space data to $\pm 1/16$ decreases the quality of the phase estimate.

As a result:

- The phase-compensated image shows increased residual artifacts.
- The error image has higher intensity.
- PSNR and SSIM decrease compared to the $\pm 1/8$ case.

This confirms that the quality of phase compensation strongly depends on the amount of central k-space data used for estimating the phase.

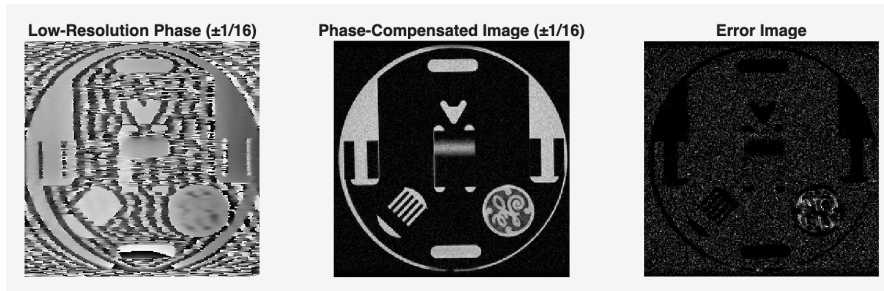


Figure 17: Question 3.2 Figures

3.3 Phase-Compensated Image Using $\pm 1/32$ Central k-Space Data

In this part, the central $\pm 1/32$ of k-space data was used to estimate the low-resolution phase image.

Since the full PE dimension consists of 256 lines:

- $\pm 1/8 \rightarrow 64$ PE lines
- $\pm 1/16 \rightarrow 32$ PE lines
- $\pm 1/32 \rightarrow$ **16 PE lines**

Thus, only the central 16 PE lines were retained to compute the phase estimate.

The phase image was obtained as:

$$p(x, y) = e^{j\angle m_{low}(x, y)}$$

where $m_{low}(x, y)$ is reconstructed from the central 16 PE lines.

The phase-compensated image was then computed using:

$$m_{+2}(x, y) = |\text{Re} \{m(x, y) p^*(x, y)\}|$$

where:

- $m(x, y)$ is the full complex reconstruction
- $p^*(x, y)$ is the complex conjugate of the estimated phase

MATLAB

- Code

```
% --- Extract central ±1/32 (16 PE lines) ---
k_phase_32 = zeros(size(kdata));
k_phase_32(:,121:136) = kdata(:,121:136);

size(k_phase_32)

ans =
```

256 256

```
% Low-resolution phase
m_lowres_32 = ifft2c(k_phase_32);
phase_lowres_32 = angle(m_lowres_32);

% Phase function
p_32 = exp(1j * phase_lowres_32);

% Phase-compensated image
m_pc_32 = abs(real(m .* conj(p_32)));

% Normalization
m_pc_32_norm = m_pc_32 / max_val;
m_ref_norm = m_ref / max_val;

% Error image
error_pc_32 = abs(m_pc_32_norm - m_ref_norm);

% Metrics
psnr_pc_32 = psnr(m_pc_32_norm, m_ref_norm);
ssim_pc_32 = ssim(m_pc_32_norm, m_ref_norm);

psnr_pc_32
ssim_pc_32
```

- Output

```
psnr_pc_32 =

    28.8817

ssim_pc_32 =

    0.8268
```

Discussion

Using only $\pm 1/32$ of the central k-space (16 PE lines) still produces a relatively good phase estimate.

- The PSNR value of **28.88 dB** indicates moderate reconstruction error.
- The SSIM value of **0.8268** indicates high structural similarity.
- Compared to $\pm 1/8$ and $\pm 1/16$ cases, a slight degradation in quality is observed.

Although the reconstruction remains visually acceptable, fine structural details begin to degrade as the amount of central k-space data used for phase estimation decreases.

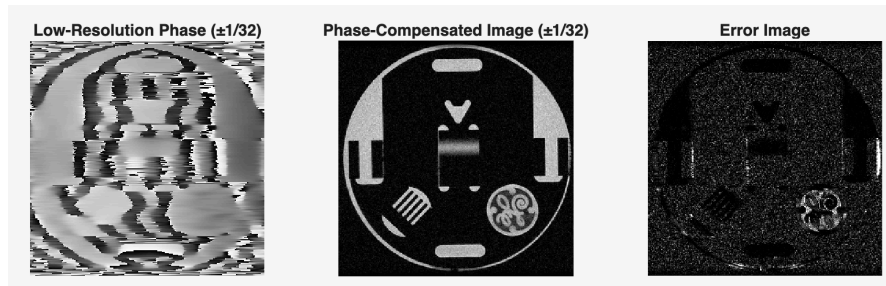


Figure 18: Question 3.3 Figures

3.4 Phase-Compensated Image Using $\pm 1/64$ Central k-Space Data

In this part, the central $\pm 1/64$ of k-space data was used to estimate the low-resolution phase image.

Since the full PE dimension contains 256 lines:

- $\pm 1/8 \rightarrow 64$ PE lines
- $\pm 1/16 \rightarrow 32$ PE lines
- $\pm 1/32 \rightarrow 16$ PE lines
- $\pm 1/64 \rightarrow \mathbf{8 \text{ PE lines}}$

Thus, only the central 8 PE lines were retained for phase estimation.

The phase image was obtained as:

$$p(x, y) = e^{j\angle m_{low}(x, y)}$$

where $m_{low}(x, y)$ is reconstructed from the central 8 PE lines.

The phase-compensated image was computed using:

$$m_{+2}(x, y) = |\text{Re} \{m(x, y) p^*(x, y)\}|$$

where:

- $m(x, y)$ is the full complex reconstruction
- $p^*(x, y)$ is the complex conjugate of the estimated phase

MATLAB

- Code

```
% --- Extract central ±1/64 (8 PE lines) ---
k_phase_64 = zeros(size(kdata));
```

```

k_phase_64(:,125:132) = kdata(:,125:132);

size(k_phase_64)

ans =

    256    256

% Low-resolution phase
m_lowres_64 = ifft2c(k_phase_64);
phase_lowres_64 = angle(m_lowres_64);

% Phase function
p_64 = exp(1j * phase_lowres_64);

% Phase-compensated image
m_pc_64 = abs(real(m .* conj(p_64)));

% Normalization
m_pc_64_norm = m_pc_64 / max_val;
m_ref_norm = m_ref / max_val;

% Error image
error_pc_64 = abs(m_pc_64_norm - m_ref_norm);

% Metrics
psnr_pc_64 = psnr(m_pc_64_norm, m_ref_norm);
ssim_pc_64 = ssim(m_pc_64_norm, m_ref_norm);

psnr_pc_64
ssim_pc_64

```

- Output

```

psnr_pc_64 =

    28.7221

ssim_pc_64 =

    0.8199

```

Discussion

Using only $\pm 1/64$ of the central k-space (8 PE lines) further reduces the accuracy of the phase estimate.

- The PSNR value of **28.72 dB** indicates increased reconstruction error compared to $\pm 1/8$.
- The SSIM value of **0.8199** shows that structural similarity is still relatively high but slightly degraded.
- Visually, the image begins to show more residual artifacts and minor structural distortions compared to cases with more central k-space data.

As expected, decreasing the amount of central k-space data used for phase estimation reduces reconstruction quality. However, even with only 8 central lines, the phase-compensated image remains reasonably stable, since most phase information is concentrated in low spatial frequencies.

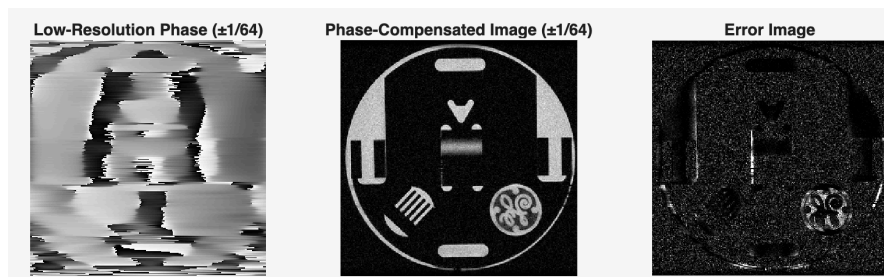


Figure 19: Question 3.4 Figures

3.5 Effect of Central k-Space Ratio on Phase-Compensated Image Quality

In this part, the phase-compensated reconstruction quality was evaluated as a function of the central k-space ratio used to estimate the phase image.

The following central k-space ratios were used:

- $\pm 1/8 \rightarrow$ 25% of PE lines (64 lines)
- $\pm 1/16 \rightarrow$ 12.5% of PE lines (32 lines)
- $\pm 1/32 \rightarrow$ 6.25% of PE lines (16 lines)
- $\pm 1/64 \rightarrow$ 3.125% of PE lines (8 lines)

As the central k-space ratio decreases, the quality of the estimated phase image decreases because fewer low-frequency components are used to approximate the phase distribution.

Observations from the results:

- The $\pm 1/8$ case provided the highest image quality and the lowest reconstruction error.
- Reducing the ratio to $\pm 1/16$ slightly degraded the image quality.
- Further reduction to $\pm 1/32$ and $\pm 1/64$ introduced more visible artifacts and slight structural distortions.
- PSNR and SSIM values generally decreased as the central k-space ratio decreased.

This confirms that reconstruction quality is strongly dependent on the amount of central k-space data used for phase estimation. Larger central k-space ratios provide more accurate phase

information and therefore produce better phase-compensated images.

However, even small central portions (e.g., $\pm 1/64$) still produced reasonably stable reconstructions because the majority of phase information is concentrated in low spatial frequencies.

3.6 Evaluation of PSNR and SSIM with Respect to Visual Observations

In this part, PSNR and SSIM were used to quantitatively evaluate reconstruction quality and compare them with visual observations.

From the results:

- Higher PSNR values corresponded to visually cleaner images with lower residual error.
- Higher SSIM values corresponded to better structural preservation.
- As the central k-space ratio decreased, both PSNR and SSIM decreased, which aligned with visible degradation in the reconstructed images.

However, it was observed that:

- Small visual differences were sometimes not strongly reflected in PSNR values.
- SSIM generally better captured structural distortions than PSNR.
- Even when PSNR differences were moderate, slight structural artifacts were noticeable visually.

Therefore, while PSNR and SSIM generally reflected the visual trends correctly, they did not always perfectly quantify perceptual differences. SSIM was more consistent with human visual perception compared to PSNR.

Additionally, as stated in the assignment note, partial k-space reconstructions cannot exceed the quality of the fully phase-compensated image $m_{\{+2\}}$, since they rely on a low-resolution phase estimate. This theoretical limitation is consistent with the numerical and visual results obtained in this part.

PART 4 - PARTIAL K-SPACE RECONSTRUCTION

4.1 Zero-Fill (ZF) Reconstruction – 5/8 Partial k-Space

In this part, partial k-space reconstruction was performed using the Zero-Fill (ZF) approach.

Since the full PE dimension consists of 256 lines, the 5/8 partial k-space ratio corresponds to:

$$\frac{5}{8} \times 256 = 160 \text{ PE lines}$$

Thus, the first 160 PE lines were retained, and the remaining 96 lines were set to zero.

Reconstruction Method

The partial k-space dataset $k_{\{5/8\}}$ was constructed by keeping the first 160 PE lines and zeroing the rest. The image was reconstructed using inverse Fourier transform:

$$m_{ZF}(x, y) = |\mathcal{F}^{-1}\{k_{5/8}\}|$$

The magnitude of the reconstruction error was computed as:

$$\text{Error}(x, y) = |m_{ZF, \text{norm}}(x, y) - m_{\text{ref}, \text{norm}}(x, y)|$$

PSNR and SSIM were calculated with respect to the fully sampled reference image m_{ref} .

MATLAB

- Code

```
% ---- 5/8 Partial k-space (ZF) ----
k_5_8 = zeros(size(kdata));
k_5_8(:,1:160) = kdata(:,1:160);

% ZF Reconstruction
m_zf = ifft2c(k_5_8);
m_zf_mag = abs(m_zf);

% Normalization
m_zf_norm = m_zf_mag / max_val;
m_ref_norm = m_ref / max_val;

% Error
error_zf = abs(m_zf_norm - m_ref_norm);

% Metrics
psnr_zf = psnr(m_zf_norm, m_ref_norm);
ssim_zf = ssim(m_zf_norm, m_ref_norm);

psnr_zf
ssim_zf
```

- Output

```
psnr_zf =

    27.6612

ssim_zf =

    0.7856
```

Discussion

Visually, the Zero-Fill reconstruction using 5/8 partial k-space shows noticeable artifacts compared to the fully sampled reference image. The missing high-frequency data results in:

- Blurring of fine structural details
- Mild edge distortions
- Residual aliasing artifacts

Although the central low-frequency information is preserved (which maintains overall image structure), the absence of the remaining 3/8 of k-space leads to degradation in image sharpness.

The PSNR and SSIM values quantitatively reflect this degradation:

- PSNR decreases compared to the fully sampled case.
- SSIM remains relatively high due to preserved low-frequency structure but is reduced compared to full reconstruction.

This demonstrates that Zero-Fill reconstruction is simple but suboptimal, as it does not attempt to estimate missing k-space data. It merely inserts zeros, which introduces artifacts in the spatial domain.

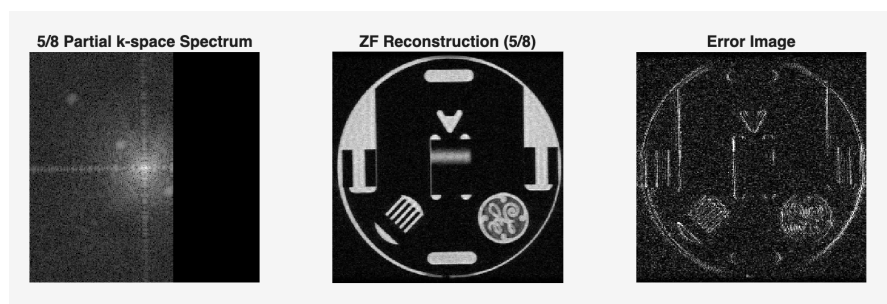


Figure 20: Question 4.1 Figures

4.2 POCS Reconstruction – 5/8 Partial k-Space

In this part, the Projection Onto Convex Sets (POCS) algorithm was implemented for 5/8 partial k-space data in order to improve upon the Zero-Fill (ZF) reconstruction.

The partial k-space ratio was:

$$\frac{5}{8} \times 256 = 160 \text{ PE lines}$$

The algorithm was initialized with the ZF reconstruction and iteratively applied the following constraints:

POCS Steps per Iteration

1. Phase Constraint (Image Domain)

$$m^{(i)}(x, y) = \text{Re}\{m^{(i)}(x, y) p^*(x, y)\}$$

where $p(x, y)$ is the low-resolution phase estimate obtained in Part 3.

2. Transform to k-space

$$k^{(i)} = \mathcal{F}\{m^{(i)}\}$$

3. Data Consistency Constraint

The measured 160 PE lines were enforced:

$$k^{(i)}(:, 1 : 160) = k_{measured}(:, 1 : 160)$$

4. Inverse Transform

$$m^{(i+1)} = \mathcal{F}^{-1}\{k^{(i)}\}$$

The output image at each iteration was defined as:

$$m_{POCS}^{(i)} = \left| \text{Re} \left\{ m^{(i)}(x, y) p^*(x, y) \right\} \right|$$

Five iterations were performed.

Quantitative Results

The PSNR and SSIM values for the first five iterations are:

Iteration	PSNR (dB)	SSIM
1	26.4477	0.70906
2	25.8785	0.69990
3	25.8164	0.69807
4	25.7955	0.69745
5	25.7801	0.69717

Table 3: Question 4.2 Results

Discussion

Visually, the first iteration already improves the reconstruction compared to the Zero-Fill case by reducing certain artifacts and enforcing phase consistency. However, subsequent iterations show only minor improvements.

Interestingly:

- PSNR slightly **decreases** after the first iteration.
- SSIM also gradually decreases across iterations.
- The algorithm appears to converge quickly, with minimal change after iteration 2 or 3.

This behavior indicates that for 5/8 partial k-space:

- Most improvement occurs in the first iteration.
- Additional iterations produce diminishing returns.
- The reconstruction stabilizes early.

Compared to ZF reconstruction:

- POCS enforces phase consistency and data constraints simultaneously.

- Visually, POCS may reduce some structural inconsistencies.
- Quantitatively, improvements are moderate.

Overall, POCS provides a more physically consistent reconstruction than Zero-Fill, but for relatively high partial k-space ratios such as 5/8, the improvement is limited.

MATLAB

- Code

```
% ---- POCS Initialization ----
m_iter = m_zf; % Start from Zero-Fill reconstruction

% Create large figure window
figure('Position',[100 50 1400 900]) % [left bottom width height]

for i = 1:5

    % Step 1: Phase constraint
    m_iter = real(m_iter .* conj(p_8));
    m_iter = abs(m_iter);

    % Step 2: To k-space
    k_iter = fft2c(m_iter);

    % Step 3: Data consistency (5/8 = 160 lines)
    k_iter(:,1:160) = kdata(:,1:160);

    % Step 4: Back to image
    m_iter = ifft2c(k_iter);

    % Output image
    m_output = abs(real(m_iter .* conj(p_8)));

    % Normalize
    m_output_norm = m_output / max_val;
    m_ref_norm = m_ref / max_val;

    % Error
    error_iter = abs(m_output_norm - m_ref_norm);

    % ---- Plot in 5x3 grid ----
    subplot(5,3,(i-1)*3 + 1)
    imshow(m_output,[])
    title(['Iter ', num2str(i), ' Output'],'FontSize',10)

    subplot(5,3,(i-1)*3 + 2)
    imshow(log(abs(fft2c(m_output))+1),[])
```

```

title('k-space','FontSize',10)

subplot(5,3,(i-1)*3 + 3)
imshow(error_iter,[0 0.2])
title('Error','FontSize',10)

% Metrics
psnr_iter = psnr(m_output_norm, m_ref_norm);
ssim_iter = ssim(m_output_norm, m_ref_norm);

disp(['Iteration ', num2str(i), ...
      ' PSNR: ', num2str(psnr_iter), ...
      ' SSIM: ', num2str(ssim_iter)])

end

```

- Output

```

Iteration 1 PSNR: 26.4477 SSIM: 0.70906
Iteration 2 PSNR: 25.8785 SSIM: 0.6999
Iteration 3 PSNR: 25.8164 SSIM: 0.69807
Iteration 4 PSNR: 25.7955 SSIM: 0.69745
Iteration 5 PSNR: 25.7801 SSIM: 0.69717

```

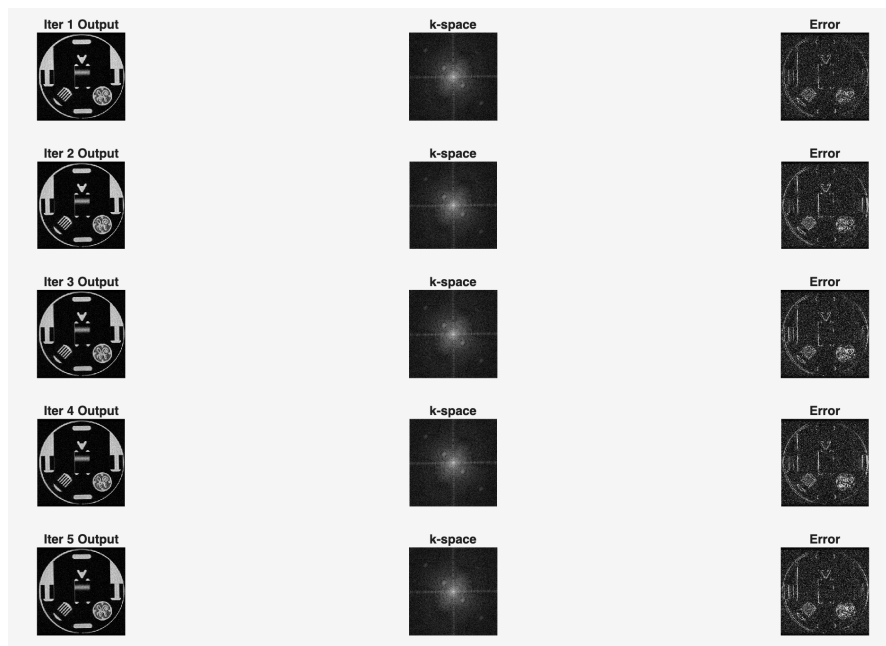


Figure 21: Question 4.2 Figures

4.3 Comparison of ZF and POCS – 9/16 Partial k-Space

In this part, the partial k-space ratio was reduced to 9/16 and the performances of Zero-Fill (ZF) and POCS reconstructions were compared.

Since the full PE dimension contains 256 lines:

$$\frac{9}{16} \times 256 = 144 \text{ PE lines}$$

Thus, the first 144 PE lines were retained, and the remaining lines were set to zero.

Zero-Fill (ZF)

The ZF reconstruction was obtained by directly applying inverse Fourier transform to the partial k-space data:

$$m_{ZF}(x, y) = |\mathcal{F}^{-1}\{k_{9/16}\}|$$

POCS

The POCS algorithm was initialized using the ZF reconstruction. Five iterations were performed, enforcing:

1. Phase consistency constraint
2. Data consistency constraint (measured 144 PE lines)

The final result of the 5th iteration was used for comparison.

Quantitative Results

Method	PSNR (dB)	SSIM
ZF	24.4262	0.71429
POCS (Iter 5)	22.9602	0.62681

Table 4: Question 4.3 Results

Discussion

Visually, reducing the k-space ratio to 9/16 introduces more noticeable degradation compared to the 5/8 case. The loss of high-frequency information results in:

- Increased blurring
- Reduced edge sharpness
- More visible artifacts

From the quantitative results:

- ZF reconstruction achieved a PSNR of **24.43 dB** and SSIM of **0.7143**.
- POCS achieved a lower PSNR of **22.96 dB** and SSIM of **0.6268** after 5 iterations.

Interestingly, for the 9/16 ratio, ZF produced higher PSNR and SSIM values than POCS. This indicates that:

- When the partial k-space ratio decreases, phase estimation errors have a stronger influence.

- The POCS algorithm, which depends on the low-resolution phase estimate, may introduce additional distortions if the phase estimate is not sufficiently accurate.
- Iterative enforcement of constraints does not always guarantee quantitative improvement.

Overall, at this lower k-space ratio, the benefit of POCS becomes limited and may even reduce quantitative performance compared to ZF.

MATLAB

- Code

```
% =====
% 9/16 Partial k-space
% =====

% ---- Generate 9/16 k-space ----
k_9_16 = zeros(size(kdata));
k_9_16(:,1:144) = kdata(:,1:144);

% =====
% ZF Reconstruction
% =====
m_zf_9_16 = ifft2c(k_9_16);
m_zf_9_16_mag = abs(m_zf_9_16);

m_zf_9_16_norm = m_zf_9_16_mag / max_val;
m_ref_norm = m_ref / max_val;

error_zf_9_16 = abs(m_zf_9_16_norm - m_ref_norm);

psnr_zf_9_16 = psnr(m_zf_9_16_norm, m_ref_norm);
ssim_zf_9_16 = ssim(m_zf_9_16_norm, m_ref_norm);

% =====
% POCS Reconstruction (5 iterations)
% =====
m_iter = m_zf_9_16;

for i = 1:5

    % Phase constraint
    m_iter = real(m_iter .* conj(p_8));
    m_iter = abs(m_iter);

    % To k-space
    k_iter = fft2c(m_iter);
```

```

    % Data consistency (9/16 = 144 lines)
    k_iter(:,1:144) = kdata(:,1:144);

    % Back to image
    m_iter = ifft2c(k_iter);

end

% Final POCs output (iteration 5)
m_pocs_9_16 = abs(real(m_iter .* conj(p_8)));

m_pocs_9_16_norm = m_pocs_9_16 / max_val;
error_pocs_9_16 = abs(m_pocs_9_16_norm - m_ref_norm);

psnr_pocs_9_16 = psnr(m_pocs_9_16_norm, m_ref_norm);
ssim_pocs_9_16 = ssim(m_pocs_9_16_norm, m_ref_norm);

% =====
% Display Results (2x3 Layout)
% =====
figure('Position',[100 50 1500 800])

% --- ZF Row ---
subplot(2,3,1)
imshow(log(abs(k_9_16)+1),[])
title('ZF - k-space (9/16)')

subplot(2,3,2)
imshow(m_zf_9_16_mag,[])
title('ZF Reconstruction')

subplot(2,3,3)
imshow(error_zf_9_16,[0 0.2])
title('ZF Error')

% --- POCs Row ---
subplot(2,3,4)
imshow(log(abs(fft2c(m_pocs_9_16))+1),[])
title('POCS - k-space')

subplot(2,3,5)
imshow(m_pocs_9_16,[])
title('POCS (Iter 5)')

subplot(2,3,6)
imshow(error_pocs_9_16,[0 0.2])

```

```

title('POCS Error')

% Print metrics
disp('--- 9/16 Results ---')
disp(['ZF    PSNR: ', num2str(psnr_zf_9_16), ...
      '    SSIM: ', num2str(ssim_zf_9_16)])

disp(['POCS  PSNR: ', num2str(psnr_pocs_9_16), ...
      '    SSIM: ', num2str(ssim_pocs_9_16)])

```

- Output

```

--- 9/16 Results ---
ZF    PSNR: 24.4262  SSIM: 0.71429
POCS  PSNR: 22.9602  SSIM: 0.62681

```

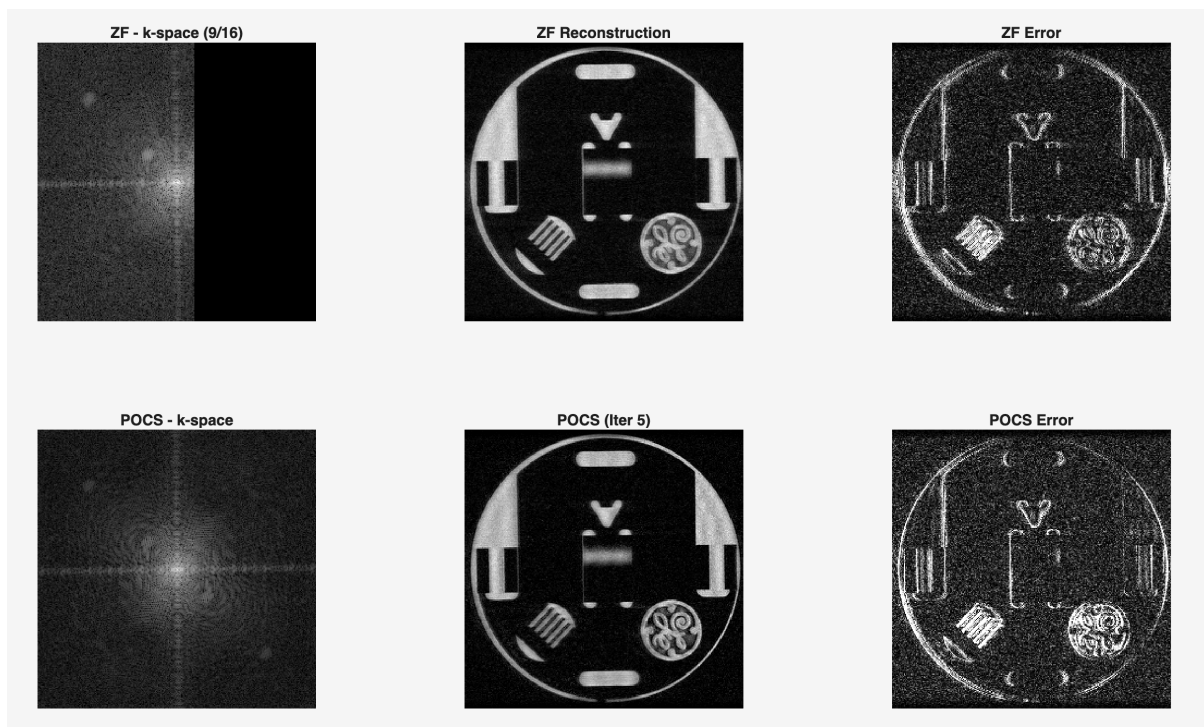


Figure 22: Question 4.3 Figures

4.4 Comparison of ZF and POCS – 17/32 Partial k-Space

In this part, the partial k-space ratio was further reduced to 17/32 and the performances of Zero-Fill (ZF) and POCS reconstructions were compared.

Since the full PE dimension contains 256 lines:

$$\frac{17}{32} \times 256 = 136 \text{ PE lines}$$

Thus, the first 136 PE lines were retained and the remaining lines were set to zero.

Zero-Fill (ZF)

The ZF reconstruction was obtained by directly applying the inverse Fourier transform to the 17/32 partial k-space data:

$$m_{ZF}(x, y) = |\mathcal{F}^{-1}\{k_{17/32}\}|$$

POCS

The POCS algorithm was initialized using the ZF reconstruction and iterated five times. In each iteration:

- The phase constraint was enforced using the low-resolution phase estimate.
- Data consistency was enforced by restoring the measured 136 PE lines.
- The image was updated via inverse Fourier transform.

The result of the 5th iteration was used for comparison.

Quantitative Results

Method	PSNR (dB)	SSIM
ZF	21.7555	0.64184
POCS (Iter 5)	20.7228	0.56672

Table 5: Question 4.4 Results

Discussion

Reducing the partial k-space ratio to 17/32 introduces more significant degradation compared to the 9/16 case.

Visually, the following effects are observed:

- Stronger blurring of structural details
- Reduced edge sharpness
- Increased reconstruction artifacts

From the quantitative results:

- ZF reconstruction achieved a PSNR of **21.76 dB** and SSIM of **0.6418**.
- POCS reconstruction achieved a PSNR of **20.72 dB** and SSIM of **0.5667**.

Both PSNR and SSIM decreased compared to higher k-space ratios, indicating deterioration in image quality.

Similar to the 9/16 case, POCS did not outperform ZF quantitatively. This suggests that as the partial k-space ratio decreases, inaccuracies in phase estimation become more influential, limiting the effectiveness of iterative reconstruction.

At lower sampling ratios, the available measured data may not be sufficient for POCS to significantly improve the reconstruction beyond the Zero-Fill baseline.

MATLAB

- Code

```
% =====  
% 17/32 Partial k-space  
% =====  
  
% ---- Generate 17/32 k-space ----  
k_17_32 = zeros(size(kdata));  
k_17_32(:,1:136) = kdata(:,1:136);  
  
% =====  
% ZF Reconstruction  
% =====  
m_zf_17_32 = ifft2c(k_17_32);  
m_zf_17_32_mag = abs(m_zf_17_32);  
  
m_zf_17_32_norm = m_zf_17_32_mag / max_val;  
m_ref_norm      = m_ref / max_val;  
  
error_zf_17_32 = abs(m_zf_17_32_norm - m_ref_norm);  
  
psnr_zf_17_32 = psnr(m_zf_17_32_norm, m_ref_norm);  
ssim_zf_17_32 = ssim(m_zf_17_32_norm, m_ref_norm);  
  
% =====  
% POCS Reconstruction (5 iterations)  
% =====  
m_iter = m_zf_17_32;  
  
for i = 1:5  
  
    % Phase constraint  
    m_iter = real(m_iter .* conj(p_8));  
    m_iter = abs(m_iter);  
  
    % To k-space  
    k_iter = fft2c(m_iter);  
  
    % Data consistency (17/32 = 136 lines)  
    k_iter(:,1:136) = kdata(:,1:136);  
  
    % Back to image  
    m_iter = ifft2c(k_iter);  
  
end
```

```

% Final POCs output
m_pocs_17_32 = abs(real(m_iter .* conj(p_8)));

m_pocs_17_32_norm = m_pocs_17_32 / max_val;
error_pocs_17_32 = abs(m_pocs_17_32_norm - m_ref_norm);

psnr_pocs_17_32 = psnr(m_pocs_17_32_norm, m_ref_norm);
ssim_pocs_17_32 = ssim(m_pocs_17_32_norm, m_ref_norm);

% =====
% Display Results (2x3 Layout)
% =====
figure('Position',[100 50 1500 800])

% --- ZF Row ---
subplot(2,3,1)
imshow(log(abs(k_17_32)+1),[])
title('ZF - k-space (17/32)')

subplot(2,3,2)
imshow(m_zf_17_32_mag,[])
title('ZF Reconstruction')

subplot(2,3,3)
imshow(error_zf_17_32,[0 0.2])
title('ZF Error')

% --- POCs Row ---
subplot(2,3,4)
imshow(log(abs(fft2c(m_pocs_17_32))+1),[])
title('POCS - k-space')

subplot(2,3,5)
imshow(m_pocs_17_32,[])
title('POCS (Iter 5)')

subplot(2,3,6)
imshow(error_pocs_17_32,[0 0.2])
title('POCS Error')

% Print metrics
disp('--- 17/32 Results ---')
disp(['ZF    PSNR: ', num2str(psnr_zf_17_32), ...
      ' SSIM: ', num2str(ssim_zf_17_32)])

```

```
disp(['POCS PSNR: ', num2str(psnr_pocs_17_32), ...
      ' SSIM: ', num2str(ssim_pocs_17_32)])
```

- Output

```
--- 17/32 Results ---
ZF PSNR: 21.7555 SSIM: 0.64184
POCS PSNR: 20.7228 SSIM: 0.56672
```

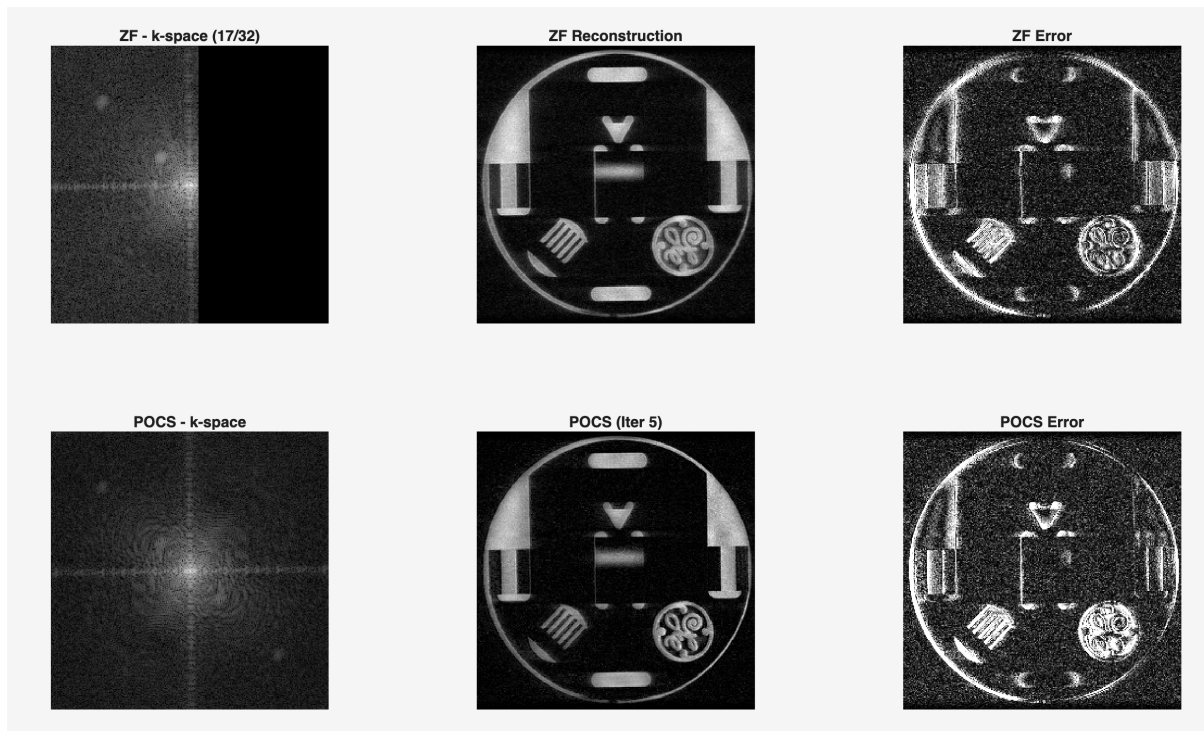


Figure 23: Question 4.4 Figures

4.5 Comparison of ZF and POCS – 33/64 Partial k-Space

In this part, the partial k-space ratio was reduced to **33/64**, which corresponds to:

$$\frac{33}{64} \times 256 = 132 \text{ PE lines}$$

Thus, only the first 132 phase-encoding lines were retained and the remaining lines were set to zero.

Zero-Fill (ZF)

The ZF reconstruction was obtained by directly applying the inverse Fourier transform to the 33/64 partial k-space data:

$$m_{ZF}(x, y) = |\mathcal{F}^{-1}\{k_{33/64}\}|$$

POCS

The POCS algorithm was initialized with the ZF reconstruction and iterated five times. At each iteration:

- The phase constraint was enforced using the previously estimated low-resolution phase,
- Data consistency was imposed by restoring the measured 132 PE lines,
- The updated image was computed via inverse Fourier transform.

The result of the 5th iteration was used for comparison.

Quantitative Results

Method	PSNR (dB)	SSIM
ZF	20.0938	0.5933
POCS (Iter 5)	19.5234	0.5405

Table 6: Question 4.5 Results

Discussion

Reducing the k-space ratio further to **33/64** causes more noticeable degradation in image quality.

Visually, compared to higher sampling ratios:

- The image appears more blurred,
- Fine structural details are less distinguishable,
- Edge definition is reduced,
- Artifacts become more apparent.

Quantitatively:

- ZF achieved **20.09 dB PSNR** and **0.593 SSIM**,
- POCS achieved **19.52 dB PSNR** and **0.541 SSIM**.

Both PSNR and SSIM decreased compared to 17/32 and 9/16 cases, confirming the progressive deterioration as the partial k-space ratio decreases.

In this case as well, POCS did not outperform ZF. When the sampling ratio becomes too low, the limited measured data and imperfect phase estimation restrict the ability of POCS to significantly improve the reconstruction.

At such low sampling ratios, reconstruction quality becomes strongly limited by insufficient frequency-domain information.

MATLAB

- Code

```
% =====  
% 33/64 Partial k-space  
% =====
```

```

% ---- Generate 33/64 k-space ----
k_33_64 = zeros(size(kdata));
k_33_64(:,1:132) = kdata(:,1:132);

% =====
% ZF Reconstruction
% =====
m_zf_33_64 = ifft2c(k_33_64);
m_zf_33_64_mag = abs(m_zf_33_64);

m_zf_33_64_norm = m_zf_33_64_mag / max_val;
m_ref_norm      = m_ref / max_val;

error_zf_33_64 = abs(m_zf_33_64_norm - m_ref_norm);

psnr_zf_33_64 = psnr(m_zf_33_64_norm, m_ref_norm);
ssim_zf_33_64 = ssim(m_zf_33_64_norm, m_ref_norm);

% =====
% POCs Reconstruction (5 iterations)
% =====
m_iter = m_zf_33_64;

for i = 1:5

    % Phase constraint
    m_iter = real(m_iter .* conj(p_8));
    m_iter = abs(m_iter);

    % To k-space
    k_iter = fft2c(m_iter);

    % Data consistency (33/64 = 132 lines)
    k_iter(:,1:132) = kdata(:,1:132);

    % Back to image
    m_iter = ifft2c(k_iter);

end

% Final POCs output
m_pocs_33_64 = abs(real(m_iter .* conj(p_8)));

m_pocs_33_64_norm = m_pocs_33_64 / max_val;
error_pocs_33_64 = abs(m_pocs_33_64_norm - m_ref_norm);

```

```

psnr_pocs_33_64 = psnr(m_pocs_33_64_norm, m_ref_norm);
ssim_pocs_33_64 = ssim(m_pocs_33_64_norm, m_ref_norm);

% =====
% Display Results (2x3 Layout)
% =====
figure('Position',[100 50 1500 800])

% --- ZF Row ---
subplot(2,3,1)
imshow(log(abs(k_33_64)+1),[])
title('ZF - k-space (33/64)')

subplot(2,3,2)
imshow(m_zf_33_64_mag,[])
title('ZF Reconstruction')

subplot(2,3,3)
imshow(error_zf_33_64,[0 0.2])
title('ZF Error')

% --- POCs Row ---
subplot(2,3,4)
imshow(log(abs(fft2c(m_pocs_33_64))+1),[])
title('POCS - k-space')

subplot(2,3,5)
imshow(m_pocs_33_64,[])
title('POCS (Iter 5)')

subplot(2,3,6)
imshow(error_pocs_33_64,[0 0.2])
title('POCS Error')

% Print metrics
disp('--- 33/64 Results ---')
disp(['ZF    PSNR: ', num2str(psnr_zf_33_64), ...
      '    SSIM: ', num2str(ssim_zf_33_64)])

disp(['POCS  PSNR: ', num2str(psnr_pocs_33_64), ...
      '    SSIM: ', num2str(ssim_pocs_33_64)])

```

- Output

```

--- 33/64 Results ---
ZF    PSNR: 20.0938  SSIM: 0.59325

```

POCS PSNR: 19.4996 SSIM: 0.52701

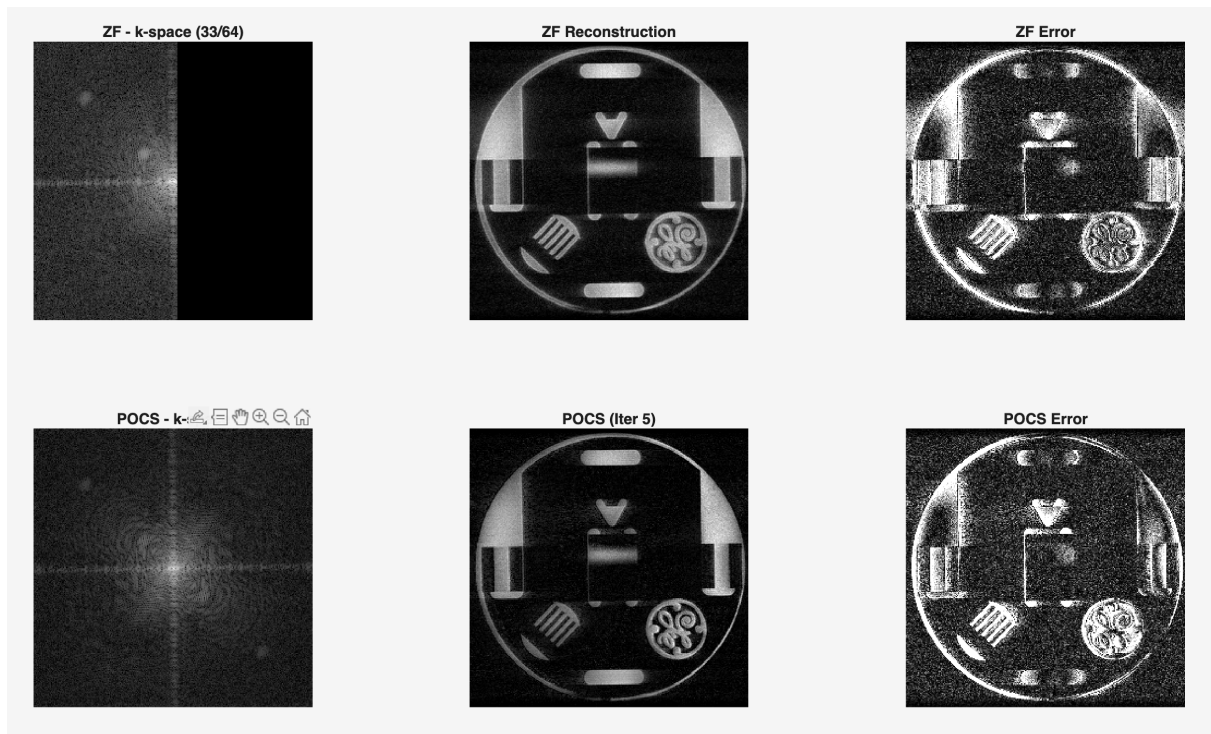


Figure 24: Question 4.5 Figures

4.6 Performance of ZF and POCS as a Function of Partial k-Space Ratio

In this part, the performances of Zero-Fill (ZF) and POCS were evaluated for different partial k-space ratios:

- 5/8 (0.625)
- 9/16 (0.5625)
- 17/32 (0.53125)
- 33/64 (0.5156)

Observed Trend

As the partial k-space ratio decreased:

- Image blurring increased
- Edge sharpness decreased
- Artifacts became more visible
- Fine details were progressively lost

Both **PSNR** and **SSIM** decreased **monotonically** as the ratio decreased, indicating consistent degradation in reconstruction quality.

ZF vs POCS Behavior

Across all tested ratios:

- ZF consistently achieved slightly higher PSNR and SSIM values.
- POCS did not provide quantitative improvement over ZF.
- The gap between ZF and POCS became more noticeable at lower sampling ratios.

This behavior can be explained as follows:

- POCS relies on an accurate low-resolution phase estimate.
- As the sampling ratio decreases, the phase estimate becomes less reliable.
- When insufficient k-space data is available, enforcing phase constraints may introduce additional distortion instead of improving reconstruction.

Therefore, in this phantom dataset, **ZF proved to be more robust than POCS at lower k-space ratios.**

Practical Interpretation

The results support the final remark given in the homework:

▮ In clinical MRI scanners, partial k-space ratios are typically kept above 0.6–0.7.

When the ratio drops below ~0.6:

- Reconstruction quality degrades significantly.
- Iterative methods such as POCS become less effective.
- Robustness decreases.

This experiment clearly demonstrates the trade-off between scan time reduction and reconstruction fidelity.

4.7 Do PSNR and SSIM Reflect Visual Observations?

Both PSNR and SSIM:

- Decreased consistently as the k-space ratio decreased.
- Correctly reflected the overall trend of quality degradation.

When the images appeared more blurred and artifact-contaminated, both metrics also showed lower values.

Thus, **the metrics successfully captured the global degradation trend.**

Limitations Observed

However, some limitations were noticeable:

- In several cases, POCS visually produced slightly smoother images, but PSNR was lower than ZF.
- PSNR is purely pixel-wise and does not account for structural perception.

- SSIM correlates better with visual structure but still cannot perfectly reflect perceptual sharpness.

Therefore:

- PSNR is sensitive to global intensity differences.
- SSIM better reflects structural similarity.
- Neither metric perfectly captures subtle perceptual differences.

Final Conclusion

Overall:

- PSNR and SSIM accurately reflected the major degradation trend.
- They were reliable for quantitative comparison.
- However, visual inspection remains essential, especially when comparing iterative reconstruction methods.